

PATRON BRIDGE – PATRON DECORATOR

OSCAR YAIR PARDO PINEDA
JORGE ALFREDO LEAL CRUZ

PRESENTADO A:

ELIECER MONTERO OJEDA

GRUPO: E196

INGENIERIA DE SISTEMAS



INSTITUCION UNIVERSITARIA TECNOLOGICA DE SANTANDER
BUCARAMANGA

2026 – 1

TABLA DE CONTENIDO

INTRODUCCION	4
OBJETIVO GENERAL	5
OBJETIVOS ESPECIFICOS	5
DESCRIPCION DEL PROYECTO	5
DIAGRAMA DE CLASES	6
ARQUITECTURA DEL SISTEMA	7
PATRONES DE DISEÑO DE SOFTWARE.....	13
PATRON SINGLETON	13
IMPLEMENTACION EN EL CODIGO	13
DIAGRAMA UML	15
PRUEBAS	15
PATRÓN FACTORY METHOD.....	16
IMPLEMENTACION EN EL CODIGO	16
DIAGRAMA UML	17
PRUEBAS	18
PATRÓN ABSTRACT FACTORY	20
IMPLEMENTACION EN EL CODIGO	20
DIAGRAMA UML	23
PRUEBAS	24
PATRON BUILDER	27
IMPLEMENTACION EN EL CODIGO	27
DIAGRAMA UML	31
PRUEBAS	32
PATRON PROTOTYPE	34
IMPLEMENTACION EN EL CODIGO	34
DIAGRAMA UML	38
PRUEBAS	38
PATRON ADAPTER.....	40
IMPLEMENTACION EN EL CODIGO	40
DIAGRAMA UML	43

PRUEBAS	44
PATRON BRIDGE.....	45
IMPLEMENTACION EN EL CODIGO	45
DIAGRAMA UML	49
PRUEBAS	50
PATRON DECORATOR	51
IMPLEMENTACION EN EL CODIGO	52
DIAGRAMA UML	56
PRUEBAS.....	57
CONCLUSIONES.....	58
REFERENCIAS BIBLIOGRAFICAS	61

INTRODUCCION

El desarrollo de software moderno requiere la aplicación de principios de diseño que permitan construir sistemas escalables, mantenibles y desacoplados. En el contexto del sector logístico y de transporte, los sistemas informáticos cumplen un papel fundamental en la gestión eficiente de flotas de vehículos, el monitoreo de operaciones y la optimización de recursos.

El presente proyecto plantea el desarrollo de un Sistema de Gestión de Flotas, orientado a administrar vehículos de transporte, controlar órdenes de operación y facilitar la toma de decisiones dentro de un entorno logístico. El sistema se apoya en una arquitectura hexagonal, la cual permite separar claramente la lógica de negocio de las dependencias tecnológicas externas, favoreciendo la mantenibilidad y evolución del sistema.

Adicionalmente, se implementan diversos patrones de diseño creacionales, tales como Singleton, Factory Method, Abstract Factory y Builder, con el propósito de mejorar la organización del código, reducir el acoplamiento entre componentes y facilitar la creación de objetos complejos dentro del dominio del sistema.

El uso de estos patrones demuestra la aplicación de buenas prácticas de ingeniería de software orientada a objetos, permitiendo construir una solución robusta que simula un entorno real de gestión logística y transporte.

OBJETIVO GENERAL

Desarrollar un Sistema de Gestión de Flotas que permita administrar vehículos y operaciones de transporte mediante el uso de una arquitectura desacoplada y la implementación de patrones de diseño que mejoren la escalabilidad, mantenibilidad y organización del software.

OBJETIVOS ESPECIFICOS

- Diseñar la arquitectura del Sistema de Gestión de Flotas utilizando el modelo de arquitectura hexagonal, con el fin de separar adecuadamente la lógica de negocio, la capa de aplicación y la infraestructura, garantizando un sistema desacoplado y mantenible.
- Implementar los módulos del sistema para la gestión de vehículos, permitiendo realizar operaciones de registro, consulta, actualización y eliminación de información mediante servicios y endpoints que soporten la administración de la flota.
- Evaluar el funcionamiento del sistema mediante pruebas sobre los servicios implementados, verificando la correcta gestión de los vehículos y la interacción entre las diferentes capas de la arquitectura del sistema.

DESCRIPCION DEL PROYECTO

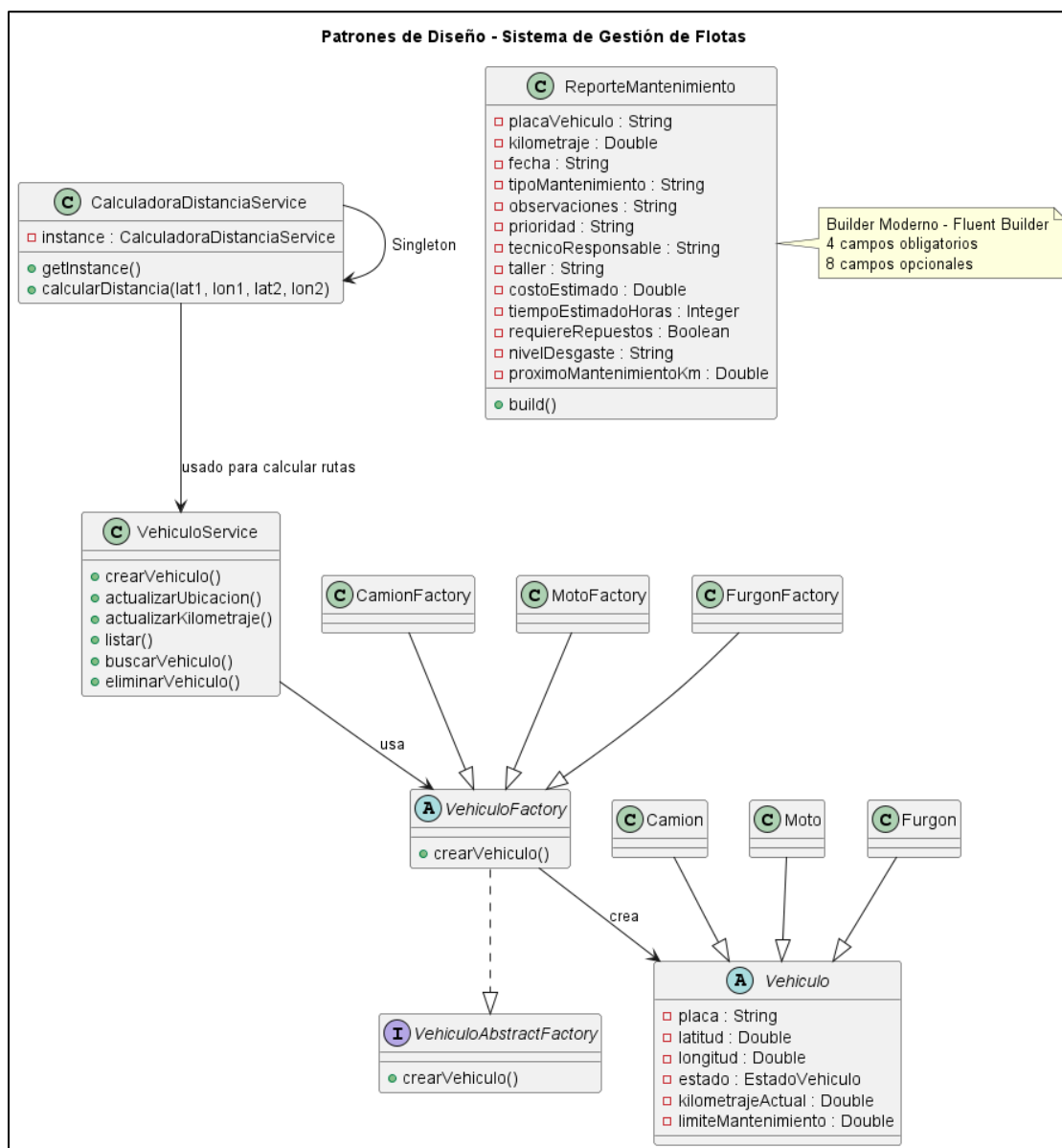


El proyecto consiste en el desarrollo de un Sistema de Gestión de Flotas enfocado en el sector logístico y de transporte, cuyo objetivo es administrar los vehículos que conforman una flota y facilitar la gestión de las operaciones de transporte.

El sistema permite realizar funciones como:

- Monitoreo en tiempo real de los vehículos registrados.
- Optimización de rutas para mejorar la eficiencia del transporte.
- Gestión y asignación de órdenes de transporte.
- Mantenimiento predictivo de los vehículos.
- Integración con sistemas de navegación.

DIAGRAMA DE CLASES



ARQUITECTURA DEL SISTEMA

La Arquitectura Hexagonal, también conocida como Arquitectura de Puertos y Adaptadores, es un modelo arquitectónico propuesto por Alistair Cockburn cuyo objetivo principal es aislar completamente la lógica de negocio de cualquier dependencia tecnológica externa.

Su principio fundamental establece que:

- El núcleo del sistema (dominio) no debe depender de frameworks, bases de datos ni interfaces externas.
- Las dependencias deben apuntar siempre hacia el interior.

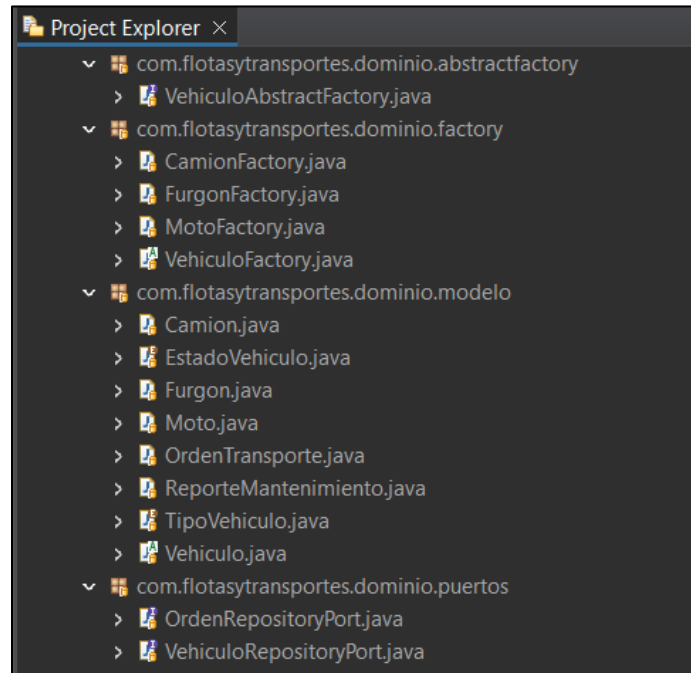
En este modelo, el dominio se ubica en el centro de la arquitectura y representa la parte más estable y crítica del sistema: las reglas del negocio. A diferencia de las arquitecturas tradicionales en capas donde el dominio suele terminar acoplado a tecnologías como frameworks web o librerías de persistencia la arquitectura hexagonal protege la lógica del negocio colocándola en el núcleo y obligando a que todo lo externo dependa de él, y no al revés.

El sistema se organiza alrededor de un núcleo (dominio) y se conecta con el mundo exterior mediante puertos y adaptadores.

- **Puertos:** Son interfaces definidas por el dominio que establecen contratos de comunicación.
 - **Puertos de entrada:** representan los casos de uso que el sistema expone.
 - **Puertos de salida:** representan servicios externos que el dominio necesita (persistencia, servicios externos, etc.).
- **Adaptadores:** Son implementaciones concretas de esos puertos. Permiten que tecnologías específicas (como controladores REST, bases de datos o APIs externas) interactúen con el sistema sin modificar el dominio.

Este enfoque permite que el sistema tenga múltiples puntos de entrada y salida (por ejemplo, API REST, consola, mensajería, base de datos), todos conectados al núcleo mediante contratos bien definidos, sin que el dominio conozca detalles técnicos.

CAPA DE DOMINIO



La capa de dominio constituye el centro arquitectónico del sistema. Está conformada por los siguientes paquetes:

- com.flotasytransportes.dominio.modelo
- com.flotasytransportes.dominio.factory
- com.flotasytransportes.dominio.puertos
- com.flotasytransportes.dominio.abstractfactory

Aquí se concentra la lógica real del negocio logístico.

➤ **MODELO**

En el paquete dominio.modelo se encuentran las entidades principales del sistema, tales como:

- Vehiculo
- Camion
- Furgon

- Moto
- OrdenTransporte
- ReporteMantenimiento
- EstadoVehiculo
- TipoVehiculo

Estas clases representan el modelo conceptual del negocio de flotas. No son simples estructuras de datos, sino que encapsulan comportamiento y reglas propias del dominio, como los estados operativos de un vehículo, la relación entre órdenes y unidades de transporte, o la clasificación por tipo.

Un aspecto fundamental es que estas clases no contienen anotaciones de Spring ni de JPA. Esto demuestra que el dominio se mantiene completamente aislado de la tecnología, cumpliendo el principio central de la arquitectura hexagonal: la lógica de negocio no depende del framework.

➤ **FACTORY**

En el paquete dominio.factory se encuentran clases como:

- VehiculoFactory
- CamionFactory
- FurgonFactory
- MotoFactory

Estas clases encapsulan la lógica de creación de los distintos tipos de vehículos. Desde el punto de vista arquitectónico, esta decisión es relevante porque la creación de entidades forma parte del negocio y no debe delegarse a la infraestructura.

Al centralizar la construcción de objetos dentro del dominio:

- Se evita la dispersión de lógica de instanciación.
- Se mantiene bajo acoplamiento.
- Se facilita la extensión del sistema ante nuevos tipos de vehículos.

➤ PORT

El paquete dominio.puertos contiene interfaces como:

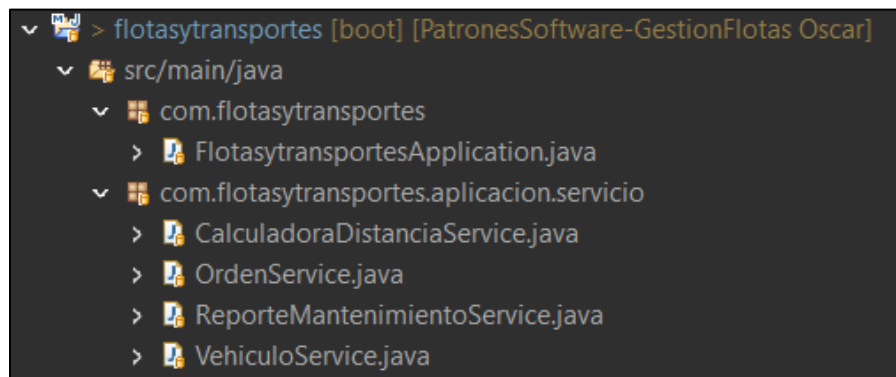
- OrdenRepositoryPort
- VehiculoRepositoryPort

Estos elementos representan los puertos de salida del sistema. Es decir, son contratos definidos por el dominio para interactuar con el exterior, particularmente con la persistencia.

El dominio declara qué necesita (guardar vehículos, consultar órdenes, persistir información), pero no define cómo se implementa. La implementación concreta se delega a la infraestructura.

Este diseño materializa el principio de Inversión de Dependencias, ya que la infraestructura depende de interfaces del dominio, y no al contrario.

CAPA DE APLICACIÓN



La capa de aplicación está representada por el paquete:

- com.flotasytransportes.aplicacion.servicio

Incluye clases como:

- VehiculoService
- OrdenService

- CalculadoraDistanciaService
- ReporteMantenimientoService

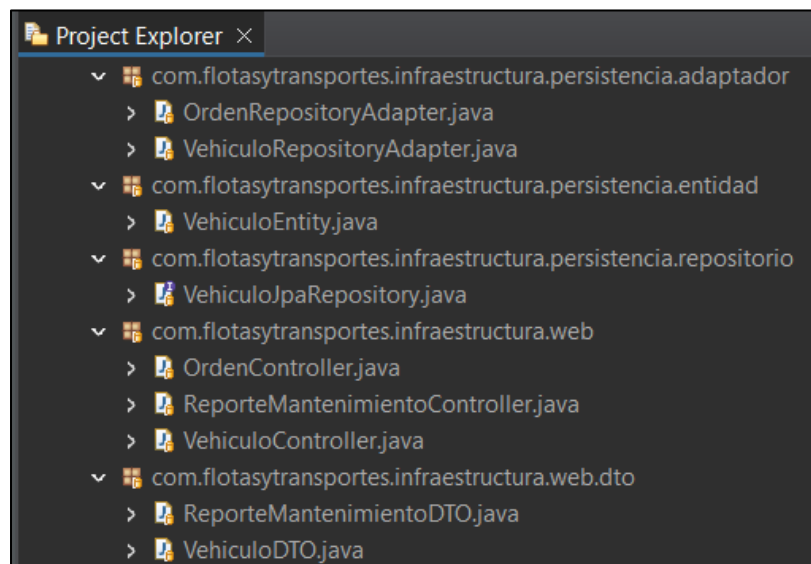
Esta capa no contiene lógica técnica ni detalles de persistencia. Su función es ejecutar los casos de uso del sistema.

Por ejemplo, un servicio puede:

1. Recibir una solicitud proveniente de la capa web.
2. Utilizar una fábrica del dominio para crear una entidad.
3. Invocar un puerto de repositorio para persistir la información.
4. Coordinar el flujo completo del caso de uso.

Es importante destacar que la capa de aplicación no es infraestructura. No maneja directamente la base de datos ni detalles de HTTP. Su responsabilidad es coordinar el dominio, manteniendo la coherencia del flujo operativo.

CAPA DE INFRAESTRUCTURA



La capa de infraestructura contiene las implementaciones técnicas que permiten conectar el dominio con el entorno externo.

➤ **ADAPTADORES**

En el paquete `infrastructure.persistencia.adaptador` se encuentran:

- `OrdenRepositoryAdapter`
- `VehiculoRepositoryAdapter`

Estas clases implementan las interfaces definidas en `dominio.puertos`. Aquí se concreta la comunicación con la base de datos.

➤ **ENTIDADES**

El paquete `infrastructure.persistencia.entidad` contiene clases como:

- `VehiculoEntity`

Es importante señalar que estas entidades son distintas a las entidades del dominio. Esto evita que el modelo del negocio quede contaminado con anotaciones técnicas, manteniendo una separación limpia entre modelo conceptual y modelo de base de datos.

➤ **REPOSITORIOS**

En `infrastructure.persistencia.repositorio` se encuentra:

- `VehiculoJpaRepository`

Aquí aparece el uso de tecnologías como Spring Data JPA y Spring Boot. Estas dependencias están correctamente ubicadas en la capa externa, cumpliendo el principio de independencia tecnológica del dominio.

PATRONES DE DISEÑO DE SOFTWARE

PATRON SINGLETON

El Singleton es un patrón de diseño creacional cuyo objetivo es garantizar que una clase tenga una única instancia durante todo el ciclo de vida de la aplicación y proporcionar un punto de acceso global controlado a dicha instancia.

El patrón se utiliza cuando:

- No tiene sentido crear múltiples instancias del mismo objeto.
- El componente coordina lógica central del sistema.
- Se desea controlar el acceso a recursos compartidos.

Tradicionalmente, en Java, se implementa mediante:

- Constructor privado.
- Atributo estático.
- Método público getInstance().

Sin embargo, en aplicaciones modernas con contenedores IoC, como **SPRING**, el patrón puede implementarse de forma gestionada por el framework.

En este proyecto, el patrón Singleton no fue implementado manualmente, sino que se utiliza el Singleton gestionado por el contenedor de Spring.

En Spring Framework, el alcance por defecto de los componentes es patrón **SINGLETON**.

Esto significa que:

- Se crea una única instancia.
- Se reutiliza en toda la aplicación.
- No se permite la creación manual repetida.



IMPORTANTE

IMPLEMENTACION EN EL CODIGO

El patrón Singleton se evidencia en las clases anotadas con la siguiente sentencia:

- @Service
- @Repository
- @Component

```

CalculadoraDistanciaService.java ×
1 package com.flotasytransportes.aplicacion.servicio;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class CalculadoraDistanciaService {
7
8     private static final double RADIO_TIERRA_KM = 6371;
9
10    public double calcularDistancia(double lat1, double lon1, double lat2, double lon2) {
11
12        double dLat = Math.toRadians(lat2 - lat1);
13        double dLon = Math.toRadians(lon2 - lon1);
14
15        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2) + Math.cos(Math.toRadians(lat1))
16            * Math.cos(Math.toRadians(lat2)) * Math.sin(dLon / 2) * Math.sin(dLon / 2);
17
18        double c = 2 * Math.asin(Math.sqrt(a));
19
20        return RADIO_TIERRA_KM * c;
21    }
22 }
23

```

```

OrdenRepositoryAdapter.java ×
1 package com.flotasytransportes.infraestructura.persistencia.adaptador;
2
3 import org.springframework.stereotype.Repository;
4
5 @Repository
6 public class OrdenRepositoryAdapter implements OrdenRepositoryPort {
7
8     @Override
9     public OrdenTransporte guardar(OrdenTransporte orden) {
10        // lógica de guardado
11        return orden;
12    }
13 }
14
15

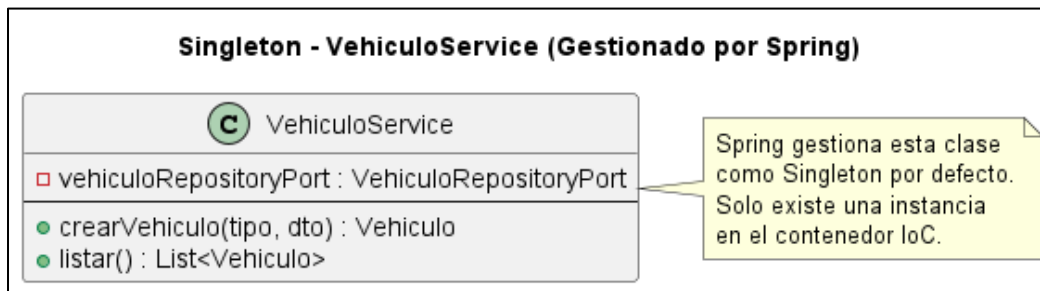
```

El patrón Singleton se utiliza en tu Sistema de Gestión de Flotas porque:

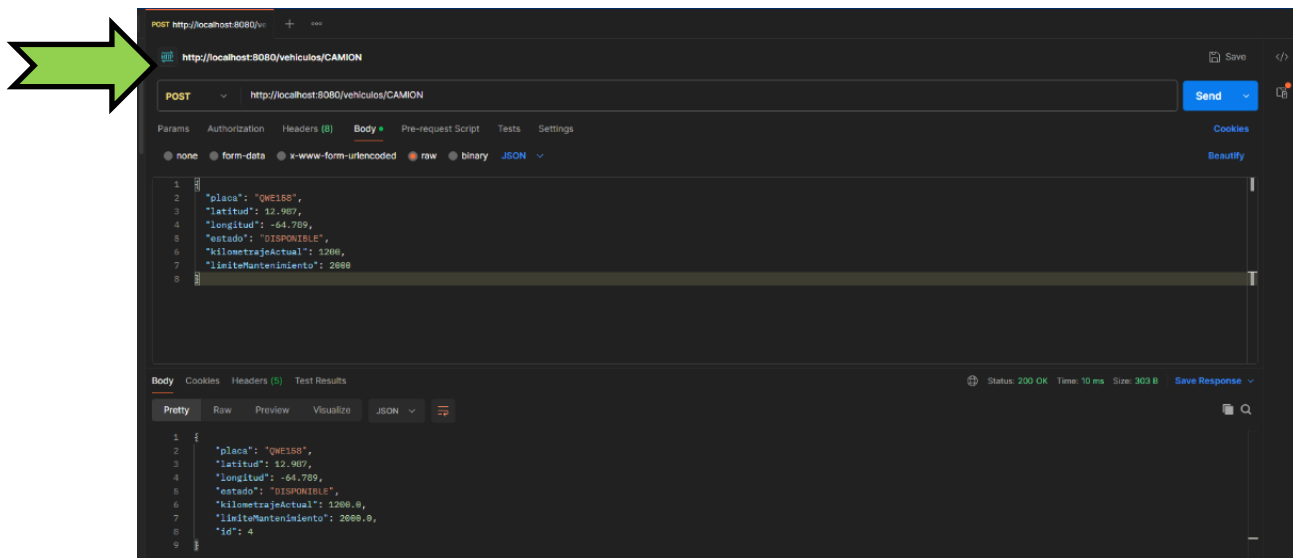
- Los servicios representan coordinadores centrales del negocio.
- Los repositorios gestionan recursos compartidos.
- No se requiere estado independiente por instancia.
- Se busca eficiencia y coherencia global.
- El framework utilizado lo implementa de manera natural y segura.

En este contexto, el Singleton no es una elección arbitraria, sino una consecuencia lógica del diseño del sistema y de la naturaleza de sus componentes.

DIAGRAMA UML



PRUEBAS



La imagen anterior muestra la ejecución de una petición HTTP al endpoint expuesto por VehiculoController, el cual utiliza internamente la clase VehiculoService, anotada con `@Service`.

Durante la ejecución de múltiples solicitudes desde Postman, se imprimió en consola el hashCode de la instancia de VehiculoService. Se evidenció que el identificador de la instancia permanece constante en cada petición, lo que confirma que el contenedor de Spring crea una única instancia de la clase y la reutiliza durante todo el ciclo de vida de la aplicación.

Este comportamiento demuestra la aplicación del patrón Singleton en la capa de servicios, ya que existe una sola instancia compartida que coordina la lógica de negocio relacionada con la gestión de vehículos.

PATRÓN FACTORY METHOD

El Factory Method es un patrón de diseño creacional cuyo propósito es definir una interfaz para la creación de objetos, permitiendo que las subclases decidan qué clase concreta instanciar.

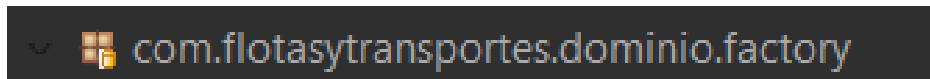
En lugar de crear objetos directamente mediante *new ClaseConcreta()* la creación se delega a una fábrica que encapsula la lógica de instanciación.

Este patrón se utiliza cuando:

- Existen múltiples variantes de un mismo tipo de objeto.
- La creación depende de una condición.
- Se desea reducir el acoplamiento entre el cliente y las clases concretas.

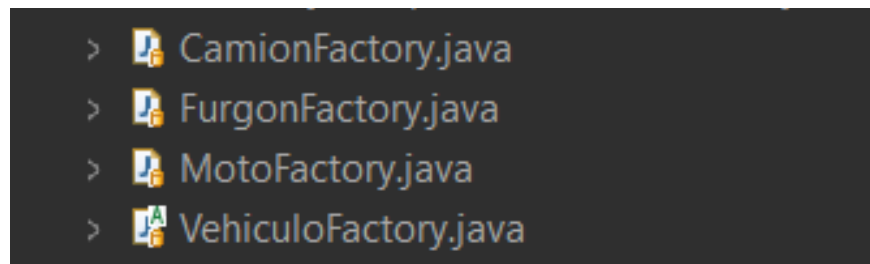
IMPLEMENTACION EN EL CODIGO

En este proyecto, el patrón Factory Method se encuentra en el paquete:



```
com.flotasytransportes.dominio.factory
```

Clases involucradas:



```
> CamionFactory.java  
> FurgonFactory.java  
> MotoFactory.java  
> VehiculoFactory.java
```

Estas clases encapsulan la lógica de creación de los distintos tipos de vehículos.

En el sistema existen distintos tipos de vehículos:

- Camión
- Furgón
- Moto

Cada uno puede tener características o comportamientos particulares.

Si la creación se hiciera directamente en el servicio con múltiples condicionales:

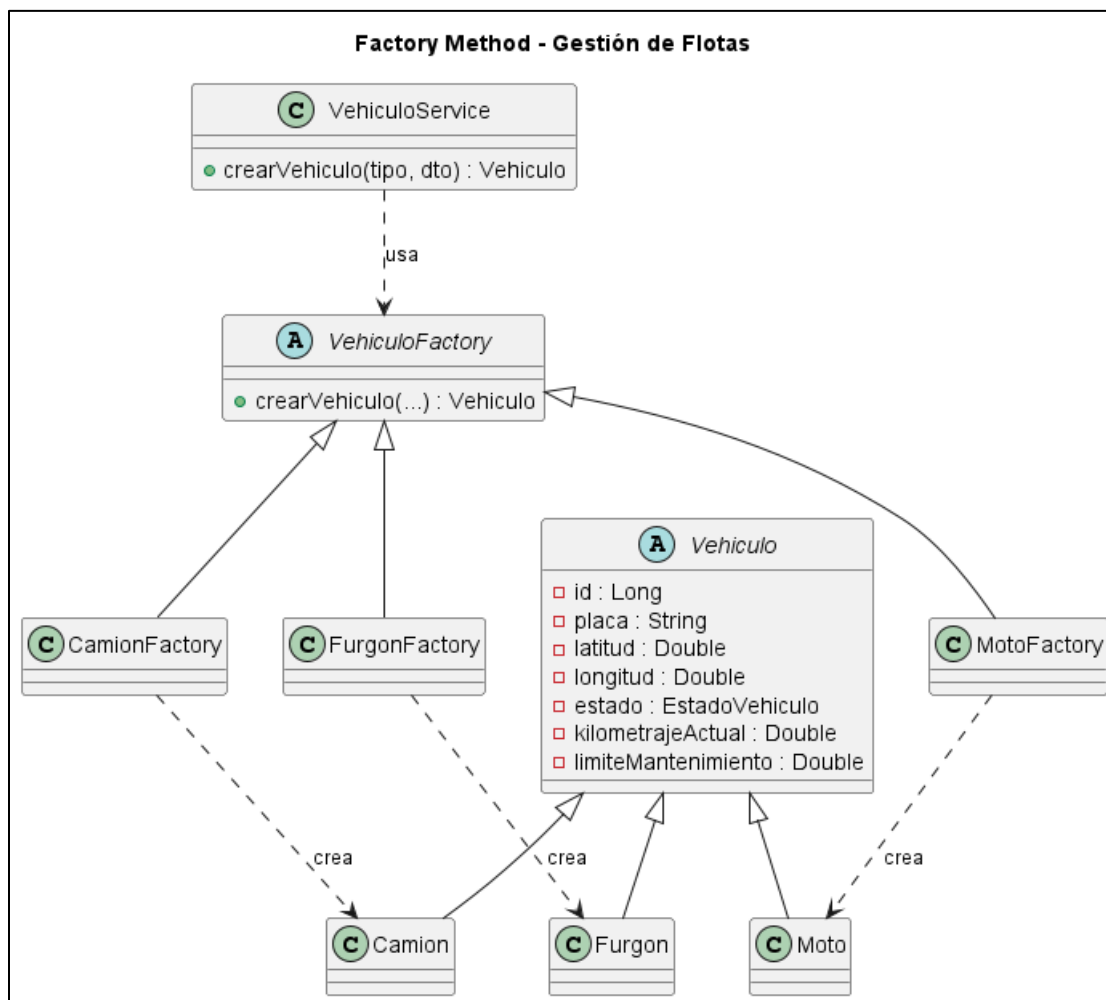
```
if(tipo.equals("CAMION")) {  
    return new Camion(...);  
} else if(tipo.equals("FURGON")) {  
    return new Furgon(...);  
}
```

Esto generaría:

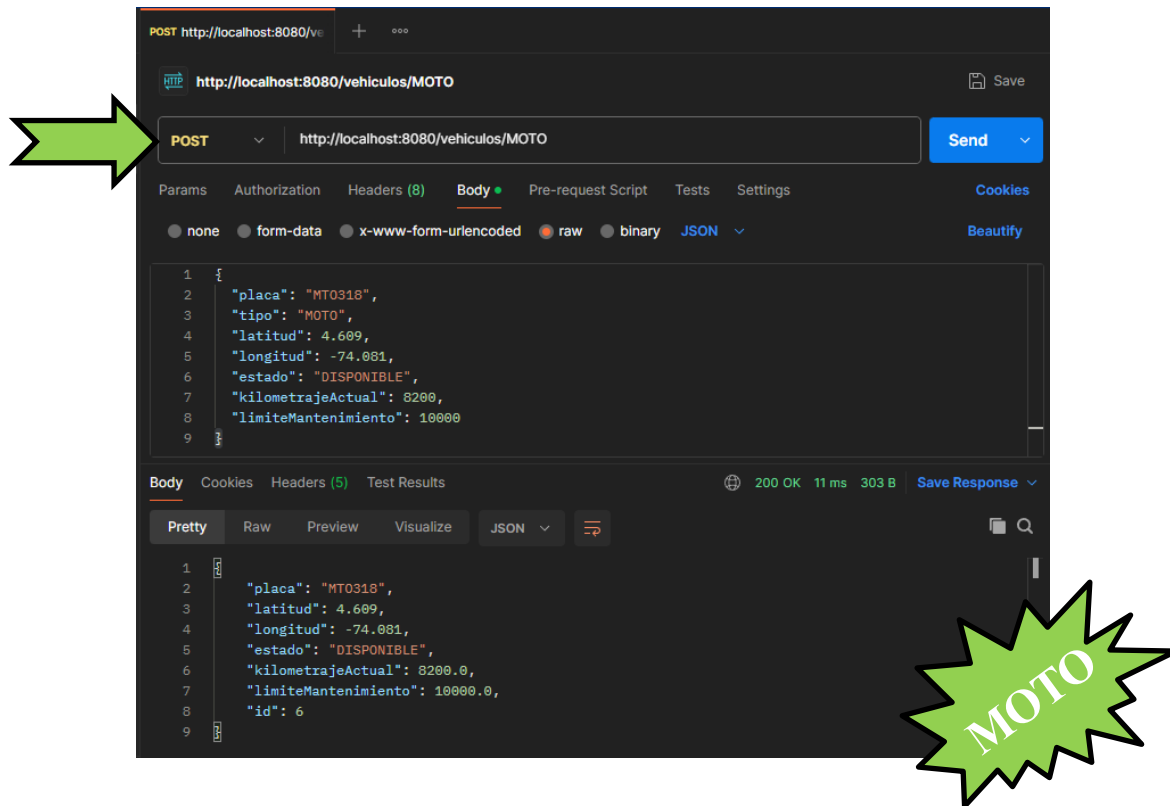
- Alto acoplamiento
- Violación del principio Open/Closed
- Código difícil de mantener
- Lógica de creación dispersa

El Factory Method centraliza esa responsabilidad.

DIAGRAMA UML



PRUEBAS



POST http://localhost:8080/vehiculos/MOTO

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

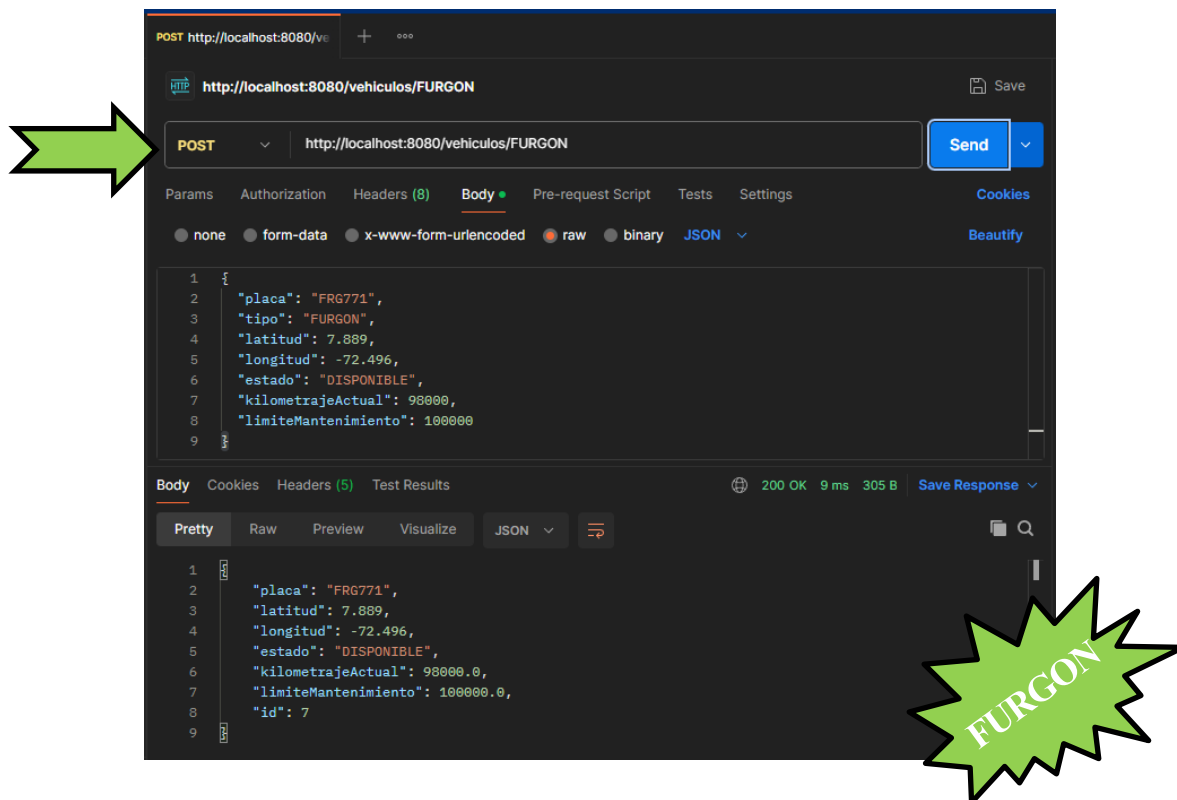
```
1 {
2   "placa": "MT0318",
3   "tipo": "MOTO",
4   "latitud": 4.609,
5   "longitud": -74.081,
6   "estado": "DISPONIBLE",
7   "kilometrajeActual": 8200,
8   "limiteMantenimiento": 10000
9 }
```

Body Cookies Headers (5) Test Results 200 OK 11 ms 303 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "placa": "MT0318",
3   "latitud": 4.609,
4   "longitud": -74.081,
5   "estado": "DISPONIBLE",
6   "kilometrajeActual": 8200.0,
7   "limiteMantenimiento": 10000.0,
8   "id": 6
9 }
```

MOTO



POST http://localhost:8080/vehiculos/FURGON

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

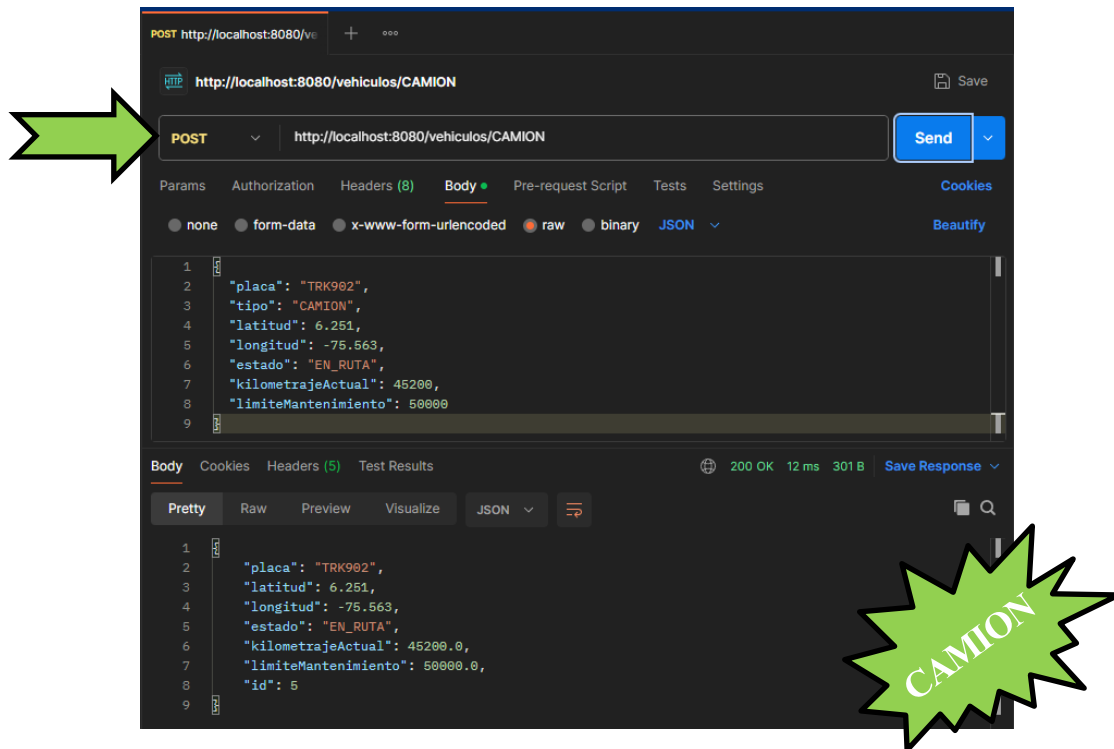
```
1 {
2   "placa": "FRG771",
3   "tipo": "FURGON",
4   "latitud": 7.889,
5   "longitud": -72.496,
6   "estado": "DISPONIBLE",
7   "kilometrajeActual": 98000,
8   "limiteMantenimiento": 100000
9 }
```

Body Cookies Headers (5) Test Results 200 OK 9 ms 305 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "placa": "FRG771",
3   "latitud": 7.889,
4   "longitud": -72.496,
5   "estado": "DISPONIBLE",
6   "kilometrajeActual": 98000.0,
7   "limiteMantenimiento": 100000.0,
8   "id": 7
9 }
```

FURGON



Con el fin de validar la correcta implementación del patrón Factory Method dentro del sistema de gestión de flotas, se realizaron pruebas de creación de distintos tipos de vehículos mediante solicitudes enviadas al endpoint correspondiente del módulo de vehículos.

Durante las pruebas se enviaron peticiones desde POSTMAN para registrar vehículos de diferentes tipos, específicamente Moto, Furgón y Camión. Cada solicitud incluye la información necesaria del vehículo, como la placa, tipo de vehículo y demás atributos definidos en el DTO utilizado por el sistema.

Cuando el sistema recibe la solicitud, la clase `VehiculoService` delega la creación del objeto a la fábrica correspondiente según el tipo de vehículo indicado. De esta manera, el sistema selecciona automáticamente la clase de fábrica adecuada (`MotoFactory`, `FurgonFactory` o `CamionFactory`), la cual se encarga de construir la instancia concreta del vehículo.

Los resultados observados en las respuestas del sistema y en los registros almacenados evidencian que cada solicitud genera correctamente el tipo de objeto esperado dentro del dominio. Esto confirma que la lógica de creación de objetos no se encuentra acoplada directamente al servicio, sino que está centralizada en las fábricas especializadas.

PATRÓN ABSTRACT FACTORY

Es un patrón de diseño creacional cuyo propósito es proporcionar una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas. Este patrón permite que el código cliente utilice diferentes conjuntos de objetos compatibles entre sí sin quedar acoplado a sus implementaciones específicas.

La idea principal del patrón es separar la lógica de creación de objetos de su uso, delegando dicha responsabilidad a una fábrica abstracta que define los métodos de creación, mientras que las fábricas concretas implementan esos métodos para instanciar los productos específicos. Este patrón es especialmente útil cuando un sistema necesita trabajar con múltiples familias de objetos que deben utilizarse conjuntamente, garantizando que los objetos creados sean compatibles entre sí.

Uno de los aspectos importantes del **ABSTRACT FACTORY** es que generalmente utiliza internamente el patrón Factory Method para realizar la instanciación de los productos concretos. Mientras que el **FACTORY METHOD** se enfoca en la creación de un solo producto, el **ABSTRACT FACTORY** coordina la creación de conjuntos completos de objetos relacionados. Entre las principales ventajas de este patrón se encuentran el desacoplamiento entre el cliente y las clases concretas, la facilidad para cambiar o ampliar familias de productos y el cumplimiento del principio Open/Closed, ya que es posible agregar nuevas familias de objetos sin modificar el código existente.

Sin embargo, también presenta algunas desventajas, como el aumento en el número de clases dentro del sistema y la dificultad para agregar nuevos tipos de productos si no fueron contemplados inicialmente en la interfaz de la fábrica. En sistemas complejos o escalables, el patrón **ABSTRACT FACTORY** resulta especialmente útil para mantener una arquitectura organizada, flexible y extensible, permitiendo que la creación de objetos se gestione de manera centralizada y coherente con el dominio del sistema.

IMPLEMENTACION EN EL CODIGO

En el proyecto se implementa el patrón **ABSTRACT FACTORY** para gestionar la creación de los distintos tipos de vehículos dentro de la flota caracterizados por su tipo de energía.

La clase abstracta definida en el dominio es:

```
VehiculoAbstractFactory.java ×
1 package com.flotasytransportes.dominio.abstractfactory;
2
3 import com.flotasytransportes.dominio.modelo.Vehiculo;
4
5
6
7 public interface VehiculoAbstractFactory {
8
9     Vehiculo crearVehiculo(String placa, Double latitud, Double longitud, EstadoVehiculo estado, TipoEnergia tipoEnergia,
10         Double kilometrajeActual, Double limiteMantenimiento);
11
12 }
```

Esta interfaz establece el contrato que deben cumplir todas las clases encargadas de crear vehículos dentro del sistema.

A partir de esta clase abstracta se implementan las siguientes clases concretas:

- CamionFactory
- FurgonFactory
- MotoFactory

Cada una de estas clases se encarga de crear un tipo específico de vehículo.

```
VehiculoFactory.java ×
1 package com.flotasytransportes.dominio.factory;
2
3 import com.flotasytransportes.dominio.abstractfactory.VehiculoAbstractFactory;
4
5
6 public abstract class VehiculoFactory implements VehiculoAbstractFactory {
7
8     // ESTE ES EL FACTORY METHOD
9     public abstract Vehiculo crearVehiculo(
10         String placa,
11         Double latitud,
12         Double longitud,
13         EstadoVehiculo estado,
14         TipoEnergia tipoEnergia,
15         Double kilometrajeActual,
16         Double limiteMantenimiento
17     );
18 }
```

En esta estructura se observa la aplicación del patrón **ABSTRACT FACTORY** para organizar la creación de los distintos tipos de vehículos del sistema. La interfaz VehiculoAbstractFactory actúa como la clase abstracta, ya que define el método crearVehiculo () que deben implementar todas las fábricas encargadas de crear objetos del dominio.

A partir de esta abstracción se implementan las clases concretas, como CamionFactory, FurgonFactory y MotoFactory. Cada una de estas clases se especializa en la creación de un tipo específico de vehículo según su tipo de energía.

Los objetos que finalmente se crean (Camión, Furgon y Moto) corresponden a los productos concretos del patrón. Todos ellos pertenecen a la misma familia de objetos del dominio (Vehículo), pero representan diferentes tipos de vehículos dentro del sistema de gestión de flotas dado que algunos su tipo de energía es ELECTRICA o mediante el gasto de GASOLINA.

```
VehiculoService.java x
60
61● public Vehiculo crearVehiculo(TipoVehiculo tipo, VehiculoDTO dto) {
62
63     VehiculoAbstractFactory abstractfactory;
64
65     switch (tipo) {
66         case CAMION:
67             abstractfactory = new CamionFactory();
68             break;
69         case MOTO:
70             abstractfactory = new MotoFactory();
71             break;
72         case FURGON:
73             abstractfactory = new FurgonFactory();
74             break;
75         default:
76             throw new IllegalArgumentException("Tipo no soportado");
77     }
78
79     Vehiculo vehiculo = abstractfactory.crearVehiculo(
80         dto.getPlaca(),
81         dto.getLatitud(),
82         dto.getLongitud(),
83         dto.getEstado(),
84         dto.getTipoEnergia(),
85         dto.getKilometrajeActual(),
86         dto.getLimiteMantenimiento()
87     );
88
89     return vehiculoRepository.guardar(vehiculo);
90 }
91
92 }
```

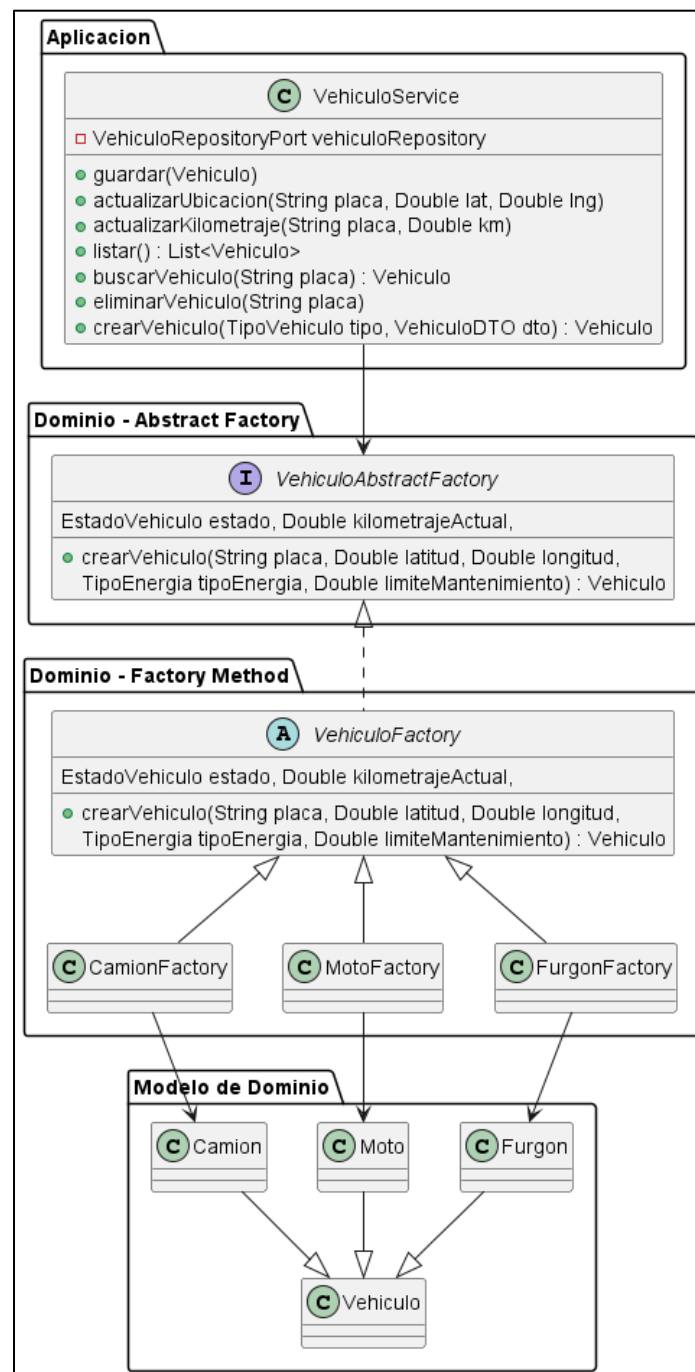
Dentro de la clase VehiculoService, este método utiliza el patrón ABSTRACT FACTORY para crear diferentes tipos de vehículos de manera desacoplada.

Primero, se declara una variable de tipo VehiculoAbstractFactory, que representa la clase abstracta encargada de definir cómo se crean los vehículos. Luego, mediante una estructura switch, el sistema selecciona la clase concreta correspondiente según el tipo de vehículo solicitado (CamionFactory, MotoFactory o FurgonFactory).

Una vez elegida la clase, se llama al método crearVehiculo () pasando los datos del VehiculoDTO. Cada clase concreta se encarga de crear el objeto específico (Camion, Moto o Furgon).

De esta forma, el servicio no depende directamente de las clases concretas, sino de la abstracción de la clase, lo que permite mantener el sistema desacoplado y facilita la incorporación de nuevos tipos de vehículos sin modificar la lógica principal.

DIAGRAMA UML

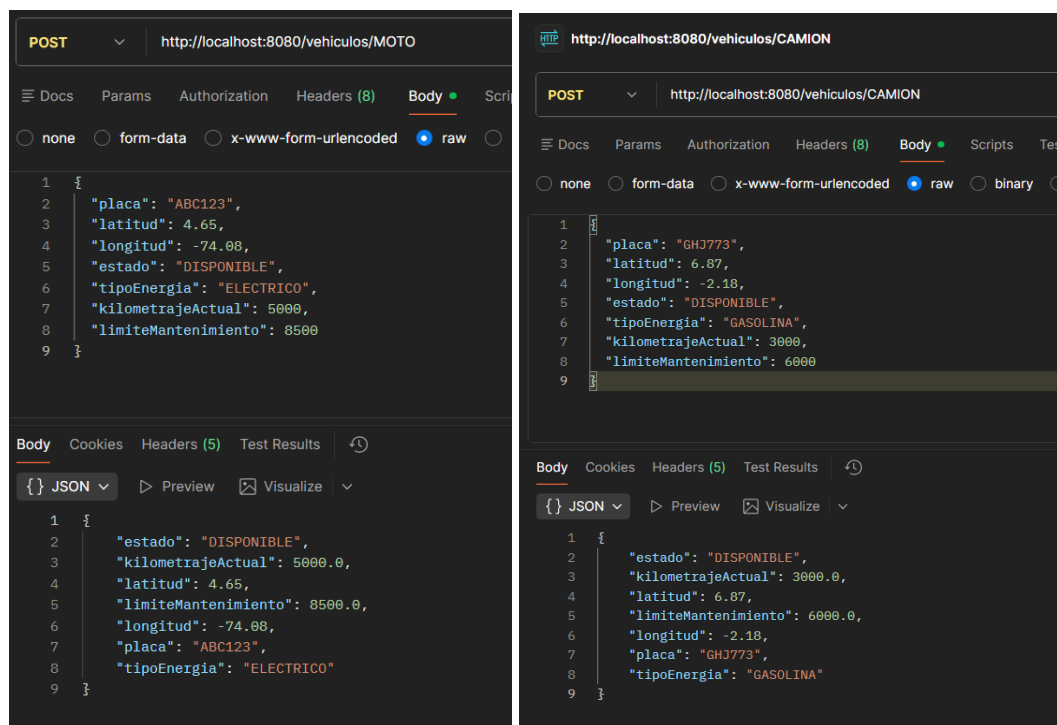


PRUEBAS

Esta sección presenta las pruebas integrales del módulo de vehículos, en el cual se implementa el patrón **ABSTRACT FACTORY** para la creación de distintos tipos de vehículos dentro del sistema. Las pruebas validan el correcto funcionamiento de las operaciones **CRUD** (Crear, Leer, Actualizar y Eliminar) utilizando el endpoint /vehículos, permitiendo gestionar vehículos a través de su placa como identificador principal.

POST / PUT	vehiculos/{placa}	Creación de un vehiculo nuevo / Actualización del vehiculo
GET	vehiculos	Lista completa de vehiculos
GET	vehiculos/{placa}	Buscar un vehiculo
DELETE	vehiculos/{placa}	Eliminar un vehiculo existente

CREACIÓN DE UN VEHICULO NUEVO



The image displays two screenshots of a REST client interface, likely Postman, showing the creation of a new vehicle via a POST request.

Left Screenshot (MOTO):

- Method: POST
- URL: `http://localhost:8080/vehiculos/MOTO`
- Body Type: raw
- Body (JSON):

```
{  "placa": "ABC123",  "latitud": 4.65,  "longitud": -74.08,  "estado": "DISPONIBLE",  "tipoEnergia": "ELECTRICO",  "kilometrajeActual": 5000,  "limiteMantenimiento": 8500}
```
- Body Tab: JSON
- Body (JSON):

```
{  "estado": "DISPONIBLE",  "kilometrajeActual": 5000.0,  "latitud": 4.65,  "limiteMantenimiento": 8500.0,  "longitud": -74.08,  "placa": "ABC123",  "tipoEnergia": "ELECTRICO"}
```

Right Screenshot (CAMION):

- Method: POST
- URL: `http://localhost:8080/vehiculos/CAMION`
- Body Type: raw
- Body (JSON):

```
{  "placa": "GHJ773",  "latitud": 6.87,  "longitud": -2.18,  "estado": "DISPONIBLE",  "tipoEnergia": "GASOLINA",  "kilometrajeActual": 3000,  "limiteMantenimiento": 6000}
```
- Body Tab: JSON
- Body (JSON):

```
{  "estado": "DISPONIBLE",  "kilometrajeActual": 3000.0,  "latitud": 6.87,  "limiteMantenimiento": 6000.0,  "longitud": -2.18,  "placa": "GHJ773",  "tipoEnergia": "GASOLINA"}
```

Se envió una solicitud POST al endpoint /vehiculos, proporcionando los datos necesarios para registrar un nuevo vehículo, incluyendo la placa, el tipo de vehículo y demás atributos requeridos. Dependiendo del tipo de energía indicado en la solicitud, el sistema utiliza el

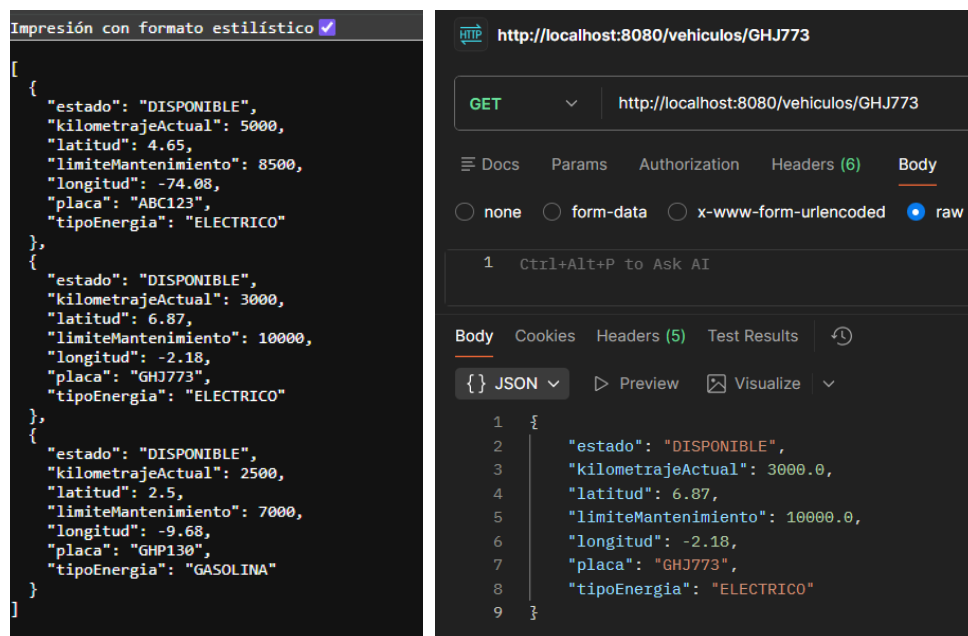
patrón Abstract Factory para seleccionar automáticamente la fábrica correspondiente, como CamionFactory, MotoFactory o FurgonFactory, encargada de crear la instancia concreta del vehículo ya sea ELECTRICO o a GASOLINA.

ACTUALIZACION DEL VEHICULO



Actualización de vehículos mediante solicitudes PUT al endpoint /vehiculos/{placa}. Estas pruebas permitieron verificar que el sistema puede modificar correctamente la información de un vehículo existente manteniendo la coherencia de los datos dentro del sistema.

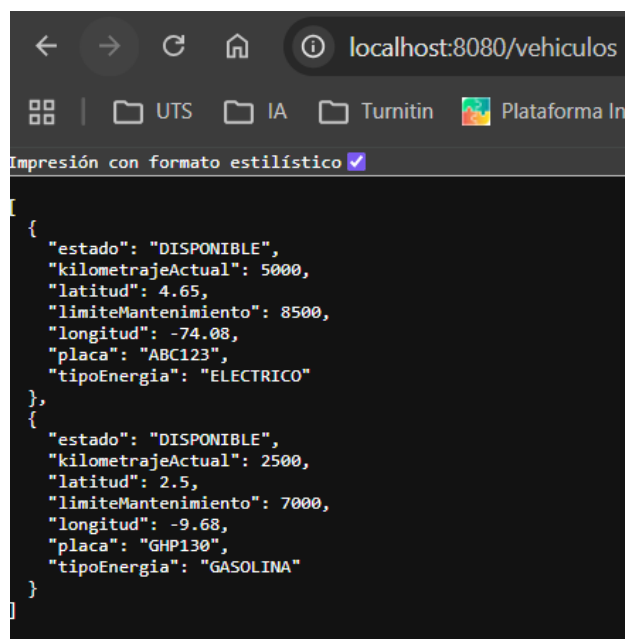
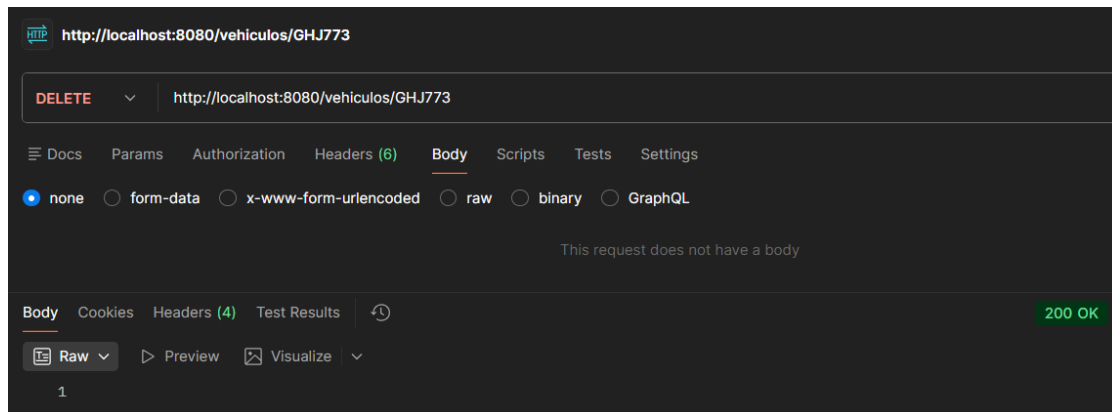
LISTA COMPLETA DE VEHICULOS



Utilizando solicitudes GET para obtener la lista completa de vehículos registrados y para buscar un vehículo específico por su placa. Los resultados mostraron que el sistema devuelve

correctamente la información almacenada, confirmando que las entidades creadas mediante las fábricas concretas se integran correctamente con la lógica del sistema.

ELIMINAR UN VEHICULO EXISTENTE



Finalmente, se realizaron pruebas de eliminación de vehículos mediante solicitudes DELETE al endpoint /vehiculos/{placa}. Estas pruebas confirmaron que el sistema puede eliminar correctamente un vehículo existente de la lista registrada.

Los resultados obtenidos en todas estas pruebas demuestran que el patrón Abstract Factory está funcionando correctamente dentro del sistema, permitiendo crear diferentes tipos de vehículos de forma desacoplada y manteniendo una arquitectura flexible y extensible.

PATRON BUILDER

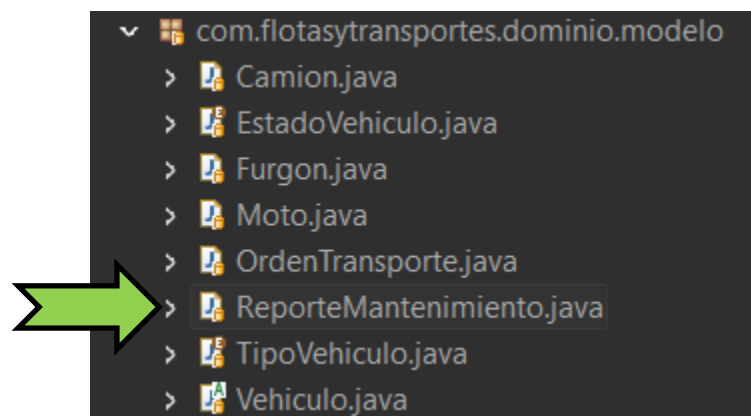
Es un patrón creacional cuyo propósito principal es construir objetos complejos paso a paso, separando el proceso de construcción de la representación final del objeto. Este patrón permite que un mismo proceso de construcción pueda crear diferentes representaciones de un objeto dependiendo de los parámetros o configuraciones utilizadas durante su creación.

En el desarrollo de software orientado a objetos, es común encontrar clases que requieren una gran cantidad de atributos para representar correctamente una entidad del sistema. Cuando una clase posee muchos atributos, especialmente cuando algunos son obligatorios y otros opcionales, el uso de constructores tradicionales puede generar código difícil de leer, mantener o extender. En estos casos, el patrón Builder proporciona una solución estructurada que permite crear objetos de manera progresiva y controlada.

El patrón Builder propone la creación de una clase constructora que se encarga de configurar los diferentes atributos del objeto final. En lugar de instanciar directamente el objeto mediante un constructor con múltiples parámetros, el desarrollador utiliza el Builder para establecer cada atributo mediante métodos específicos, y finalmente invoca un método el cual se encarga de generar la instancia completa del objeto.

IMPLEMENTACION EN EL CODIGO

El patrón Builder se implementa en la clase:



Esta clase representa un reporte de mantenimiento de un vehículo, el cual contiene múltiples atributos relacionados con el mantenimiento realizado.

```

ReporteMantenimiento.java X
38
40
41 public static class Builder {
42     // OBLIGATORIOS
43     private String placaVehiculo;
44     private Double kilometraje;
45     private String fecha;
46     private String tipoMantenimiento;
47
48     // OPCIONALES
49     private String observaciones;
50     private String prioridad;
51     private String tecnicoResponsable;
52     private String taller;
53     private Double costoEstimado;
54     private Integer tiempoEstimadoHoras;
55     private Boolean requiereRepuestos;

```

```

ReporteMantenimiento.java X
56 // CONSTRUCTOR CON OBLIGATORIOS
57 public Builder(String placaVehiculo, Double kilometraje, String fecha, String tipoMantenimiento) {
58     this.placaVehiculo = placaVehiculo;
59     this.kilometraje = kilometraje;
60     this.fecha = fecha;
61     this.tipoMantenimiento = tipoMantenimiento;
62 }
63
64
65
66 public Builder observaciones(String observaciones) {
67     this.observaciones = observaciones;
68     return this;
69 }
70
71 public Builder prioridad(String prioridad) {
72     this.prioridad = prioridad;
73     return this;
74 }
75
76 public Builder tecnicoResponsable(String tecnicoResponsable) {
77     this.tecnicoResponsable = tecnicoResponsable;
78     return this;
79 }
80
81 public Builder taller(String taller) {
82     this.taller = taller;
83     return this;
84 }
85
86 public Builder costoEstimado(Double costoEstimado) {
87     this.costoEstimado = costoEstimado;
88     return this;
89 }

```

La clase Builder es una clase interna estática dentro de ReporteMantenimiento cuya función es implementar el patrón de diseño Builder. Este patrón permite construir objetos complejos paso a paso, separando los atributos obligatorios de los opcionales. Gracias a esta estructura, el proceso de creación del objeto se vuelve más organizado, legible y flexible dentro del sistema.

Dentro del Builder se definen primero los atributos obligatorios del reporte de mantenimiento, los cuales son placaVehiculo, kilometraje, fecha y tipoMantenimiento. Estos datos son necesarios para que el sistema pueda identificar correctamente el vehículo y el tipo

de mantenimiento que se realizará. Por esta razón, estos atributos se reciben a través del constructor del Builder, garantizando que siempre estén presentes cuando se cree un reporte.

Además de los campos obligatorios, el Builder también incluye varios atributos opcionales, como observaciones, prioridad, técnico responsable, taller, costo estimado, tiempo estimado de trabajo, si requiere repuestos, nivel de desgaste y el kilometraje recomendado para el próximo mantenimiento. Estos atributos permiten agregar información adicional al reporte, pero no son indispensables para su creación.

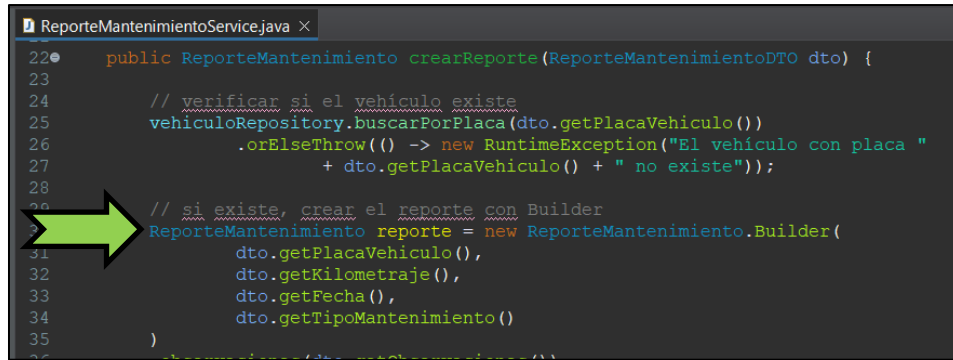


```
ReporteMantenimientoController.java x
1 package com.flotasytransportes.infraestructura.web;
2
3 import com.flotasytransportes.aplicacion.servicio.ReporteMantenimientoService;
4
5 @RestController
6 @RequestMapping("/reportes")
7 public class ReporteMantenimientoController {
8
9     private final ReporteMantenimientoService service;
10
11     public ReporteMantenimientoController(ReporteMantenimientoService service) {
12         this.service = service;
13     }
14
15     @PostMapping("/mantenimiento")
16     public ReporteMantenimiento crear(@Valid @RequestBody ReporteMantenimientoDTO dto) {
17         return service.crearReporte(dto);
18     }
19
20     @GetMapping
21     public List<ReporteMantenimiento> listar() {
22         return service.listarReportes();
23     }
24 }
25
26
27
28
29
30
31 }
```

En la clase ReporteMantenimientoController la implementación del patrón Builder no se realiza directamente, pero esta clase participa en su uso dentro del flujo de la aplicación. El controlador recibe los datos necesarios para crear un reporte de mantenimiento mediante una solicitud HTTP y los envía a la capa de servicio, donde posteriormente se utiliza el patrón Builder para construir el objeto ReporteMantenimiento.

La sección donde se relaciona con la implementación del patrón Builder es el método crear(), anotado con @PostMapping("/mantenimiento"). En este método se recibe un objeto ReporteMantenimientoDTO a través del cuerpo de la petición (@RequestBody). Este DTO contiene los datos que el usuario envía para crear el reporte de mantenimiento. Una vez recibido y validado con @Valid, el controlador envía este objeto al método crearReporte() del servicio ReporteMantenimientoService.

Es precisamente dentro del servicio donde se utiliza el Builder para construir la instancia de ReporteMantenimiento utilizando los datos del DTO. Por lo tanto, el controlador cumple el rol de punto de entrada de los datos que posteriormente serán utilizados por el Builder para crear el objeto final.



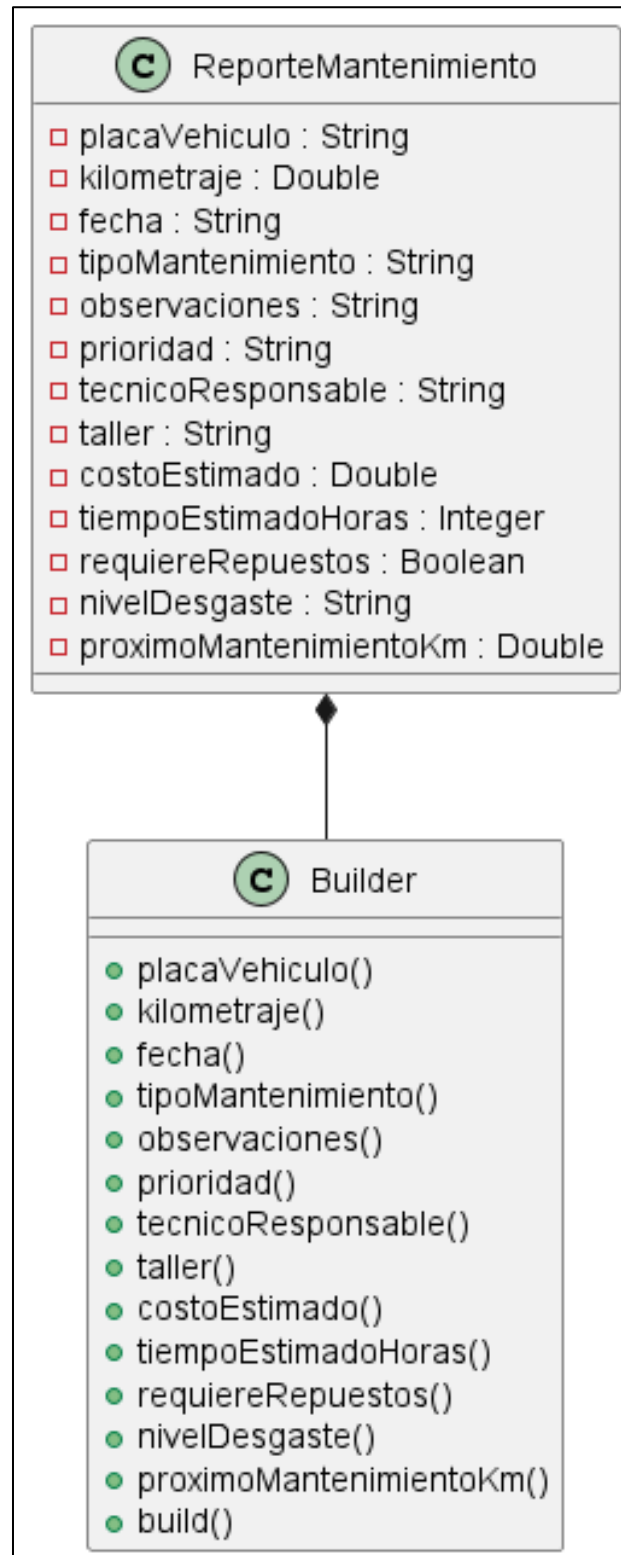
```
ReporteMantenimientoService.java x
22 public ReporteMantenimiento crearReporte(ReporteMantenimientoDTO dto) {
23
24     // verificar si el vehículo existe
25     vehiculoRepository.buscarPorPlaca(dto.getPlacaVehiculo())
26     .orElseThrow(() -> new RuntimeException("El vehículo con placa "
27     + dto.getPlacaVehiculo() + " no existe"));
28
29     // si existe, crear el reporte con Builder
30     ReporteMantenimiento reporte = new ReporteMantenimiento.Builder(
31         dto.getPlacaVehiculo(),
32         dto.getKilometraje(),
33         dto.getFecha(),
34         dto.getTipoMantenimiento()
35     )
36 }
```

En la clase ReporteMantenimientoService se utiliza directamente el patrón Builder dentro del método crearReporte(ReporteMantenimientoDTO dto). En este método primero se verifica si el vehículo existe en el sistema utilizando el repositorio VehiculoRepositoryPort. Si el vehículo no existe, se lanza una excepción para evitar crear un reporte con una placa inválida.

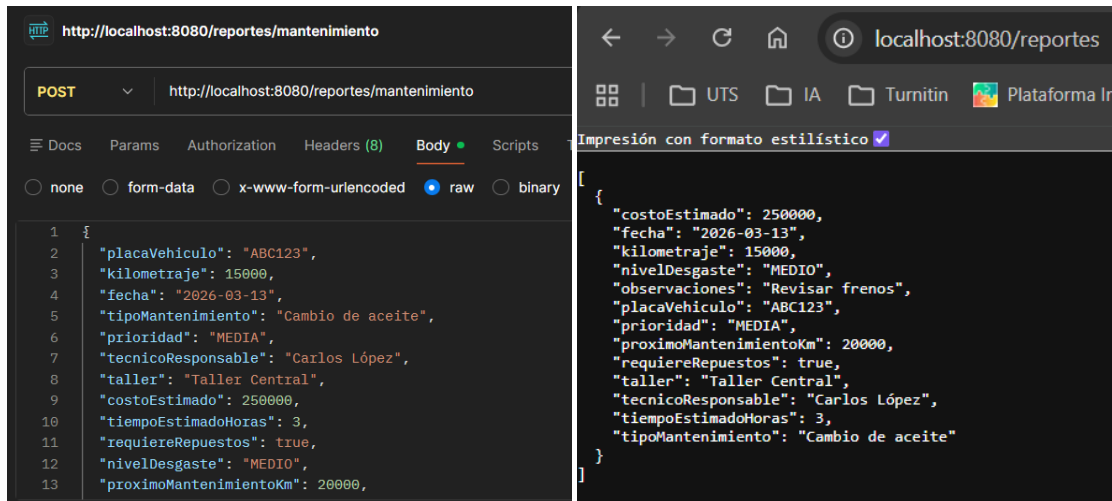
Una vez validado el vehículo, se procede a construir el objeto ReporteMantenimiento utilizando el patrón Builder mediante la instrucción new ReporteMantenimiento.Builder(...). Allí se pasan los datos obligatorios como la placa del vehículo, el kilometraje, la fecha y el tipo de mantenimiento. Luego, mediante métodos encadenados se asignan los atributos opcionales del reporte.

Finalmente, al llamar al método build() se crea el objeto ReporteMantenimiento completo, el cual se guarda en la lista de reportes y se retorna como resultado del método. De esta manera, el patrón Builder permite crear el objeto de forma más clara y organizada.

DIAGRAMA UML



PRUEBAS

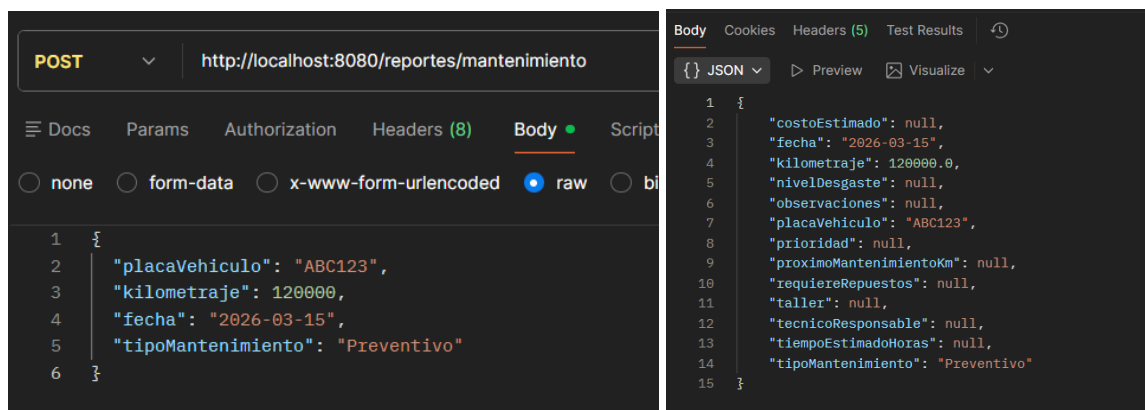


```
POST http://localhost:8080/reportes/mantenimiento

{
  "placaVehiculo": "ABC123",
  "kilometraje": 15000,
  "fecha": "2026-03-13",
  "tipoMantenimiento": "Cambio de aceite",
  "prioridad": "MEDIA",
  "tecnicoResponsable": "Carlos López",
  "taller": "Taller Central",
  "costoEstimado": 250000,
  "tiempoEstimadoHoras": 3,
  "requiereRepuestos": true,
  "nivelDesgaste": "MEDIO",
  "proximoMantenimientoKm": 20000
}
```

```
{
  "costoEstimado": 250000,
  "fecha": "2026-03-13",
  "kilometraje": 15000,
  "nivelDesgaste": "MEDIO",
  "observaciones": "Revisar frenos",
  "placaVehiculo": "ABC123",
  "prioridad": "MEDIA",
  "proximoMantenimientoKm": 20000,
  "requiereRepuestos": true,
  "taller": "Taller Central",
  "tecnicoResponsable": "Carlos López",
  "tiempoEstimadoHoras": 3,
  "tipoMantenimiento": "Cambio de aceite"
}
```

La primera prueba consistió en la creación de un reporte de mantenimiento enviando una solicitud POST con todos los datos requeridos. En esta solicitud se incluyen los atributos obligatorios del reporte, como la placa del vehículo, el kilometraje, la fecha y el tipo de mantenimiento. Además, se pueden incluir atributos opcionales como observaciones, prioridad, técnico responsable, taller, costo estimado o nivel de desgaste. Una vez recibida la solicitud, el sistema utiliza el patrón Builder dentro del servicio ReporteMantenimientoService para construir el objeto ReporteMantenimiento paso a paso y finalmente generar la instancia completa mediante el método build().



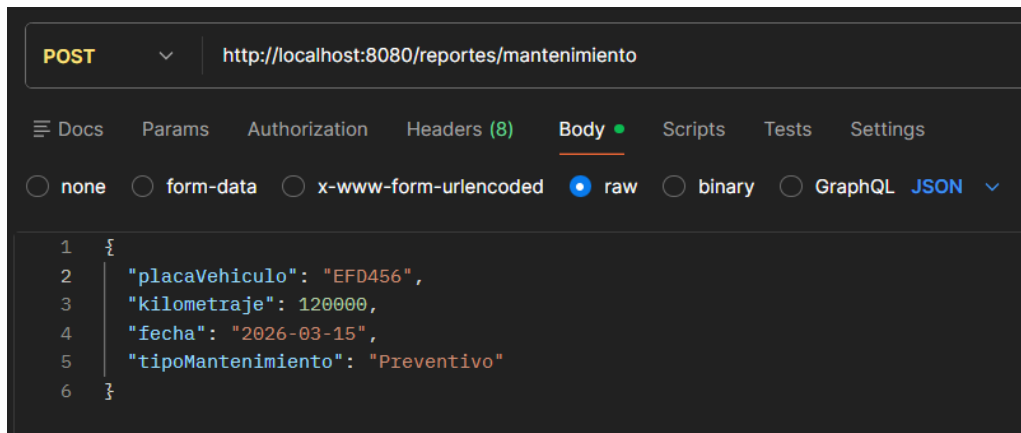
```
POST http://localhost:8080/reportes/mantenimiento

{
  "placaVehiculo": "ABC123",
  "kilometraje": 120000,
  "fecha": "2026-03-15",
  "tipoMantenimiento": "Preventivo"
}
```

```
{
  "costoEstimado": null,
  "fecha": "2026-03-15",
  "kilometraje": 120000,
  "nivelDesgaste": null,
  "observaciones": null,
  "placaVehiculo": "ABC123",
  "prioridad": null,
  "proximoMantenimientoKm": null,
  "requiereRepuestos": null,
  "taller": null,
  "tecnicoResponsable": null,
  "tiempoEstimadoHoras": null,
  "tipoMantenimiento": "Preventivo"
}
```

Otra de las pruebas realizadas fue la creación de un reporte sin diligenciar algunos de los atributos opcionales. En este caso, el sistema permitió crear el reporte correctamente, lo cual demuestra el funcionamiento del patrón Builder, ya que permite construir objetos complejos

separando los atributos obligatorios de los opcionales, evitando la necesidad de utilizar constructores extensos con múltiples parámetros.



```
POST http://localhost:8080/reportes/mantenimiento

Docs Params Authorization Headers (8) Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1 {
2   "placaVehiculo": "EFD456",
3   "kilometraje": 120000,
4   "fecha": "2026-03-15",
5   "tipoMantenimiento": "Preventivo"
6 }
```

```
java.lang.RuntimeException: El vehículo con placa EFD456 no existe
```

Adicionalmente, se realizó una prueba en la cual se intentó crear un reporte de mantenimiento utilizando una placa de vehículo que no se encuentra registrada en la base de datos. En este caso, el sistema realiza una validación previa dentro del servicio `ReporteMantenimientoService`, consultando el repositorio de vehículos. Si el vehículo no existe, se genera una excepción y el sistema no permite crear el reporte. Esta validación garantiza la integridad de la información, evitando que se registren mantenimientos asociados a vehículos inexistentes.

PATRON PROTOTYPE

El patrón Prototype es un patrón de diseño creacional cuyo propósito es crear nuevos objetos a partir de la clonación de una instancia existente, evitando la creación directa mediante constructores tradicionales. En lugar de instanciar objetos desde cero, el sistema utiliza un objeto base (prototipo) que contiene una configuración inicial, el cual puede ser copiado para generar nuevas instancias.

Este patrón resulta especialmente útil cuando la creación de objetos es costosa en términos de tiempo o recursos, o cuando los objetos poseen múltiples configuraciones que se repiten frecuentemente dentro del sistema. Mediante la clonación, se reduce la complejidad del proceso de creación y se mejora la eficiencia del sistema.

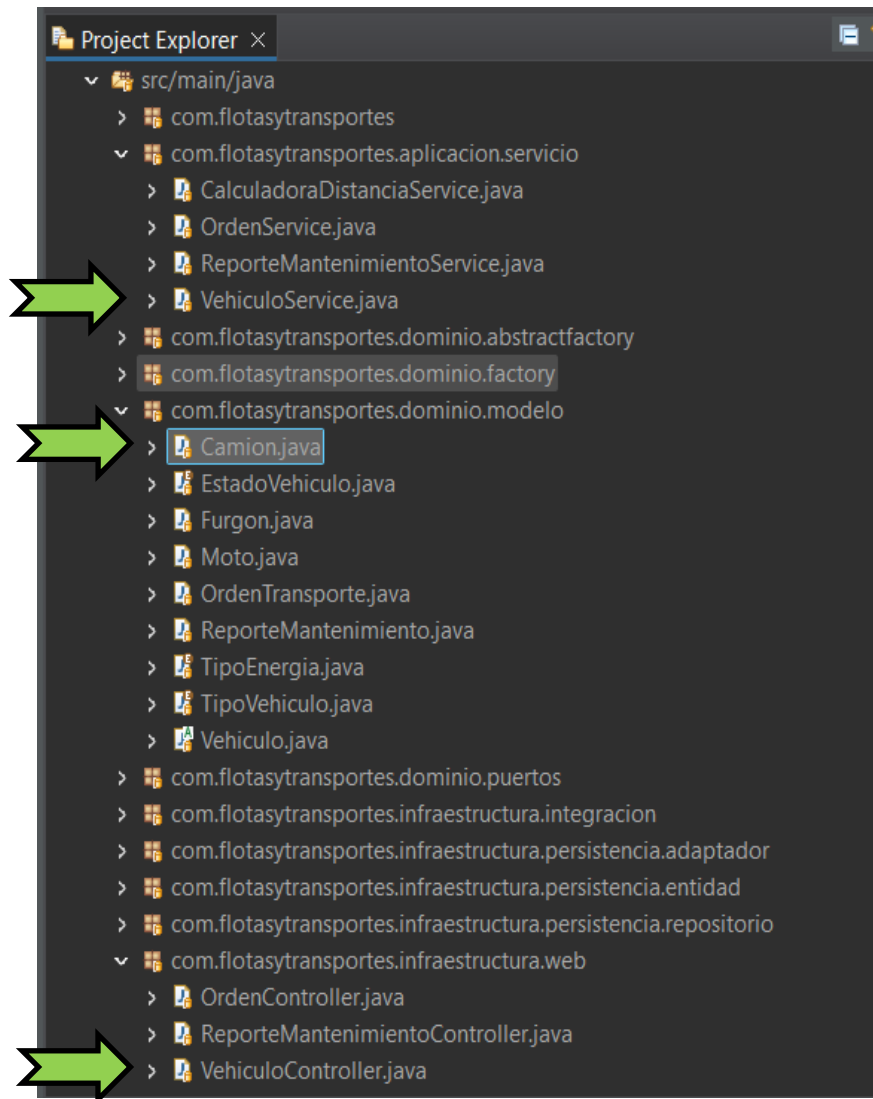
Desde el punto de vista estructural, el patrón Prototype suele apoyarse en la implementación de un método como `clone()`, el cual permite duplicar el objeto manteniendo sus atributos internos. Dependiendo del caso, la clonación puede ser superficial (shallow copy) o profunda (deep copy), garantizando que las nuevas instancias sean independientes del objeto original.

En el contexto del sistema de gestión de flotas, el patrón Prototype permite replicar objetos del dominio, como vehículos o reportes de mantenimiento, que ya han sido previamente configurados. Por ejemplo, es posible clonar un vehículo existente con ciertas características operativas y generar una nueva instancia modificando únicamente atributos como la placa o el estado.

De esta manera, el sistema evita la recreación completa del objeto y facilita la reutilización de configuraciones existentes, lo cual mejora la productividad del desarrollo y mantiene la coherencia en la creación de entidades dentro del dominio.

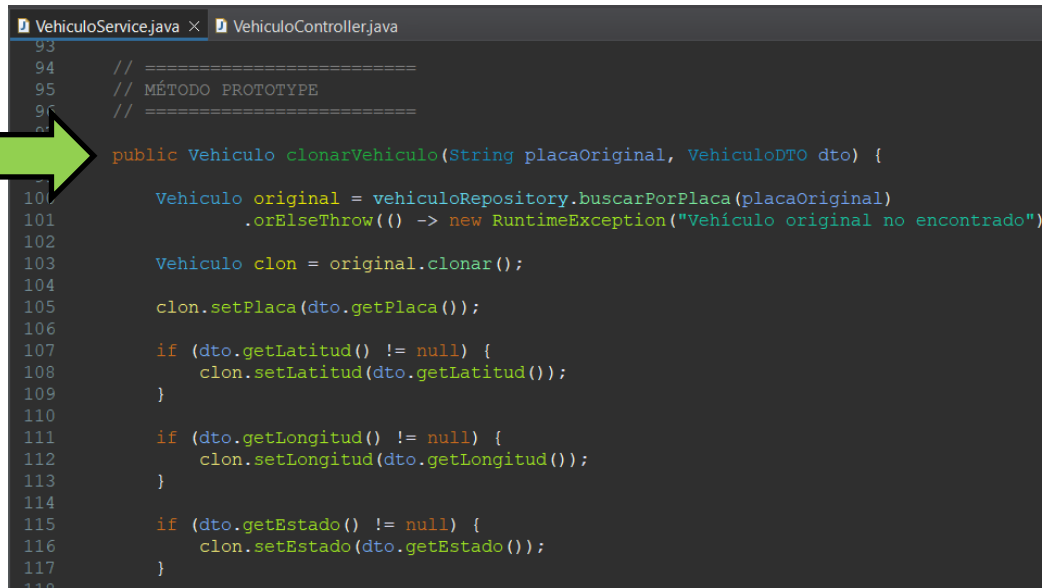
IMPLEMENTACION EN EL CODIGO

El patrón Prototype se implementa en la clase:



En la imagen se observan las clases involucradas en la implementación del patrón Prototype dentro del sistema. Se identifica la clase principal del dominio, la cual actúa como prototipo y contiene el método de clonación que permite generar nuevas instancias a partir de un objeto existente.

Esta estructura evidencia cómo el sistema reutiliza configuraciones previamente definidas, permitiendo la creación de nuevos objetos de manera eficiente sin necesidad de instanciarlos desde cero, lo cual reduce el acoplamiento y mejora la organización del código.



```

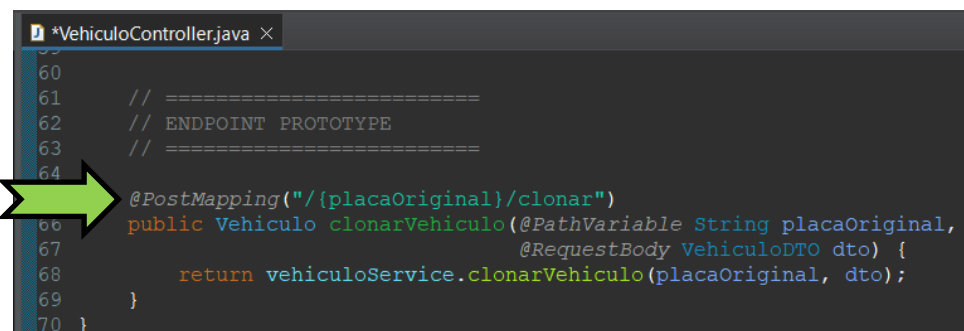
93
94 // =====
95 // MÉTODO PROTOTYPE
96 // =====
97
98 public Vehiculo clonarVehiculo(String placaOriginal, VehiculoDTO dto) {
99
100     Vehiculo original = vehiculoRepository.buscarPorPlaca(placaOriginal)
101         .orElseThrow(() -> new RuntimeException("Vehículo original no encontrado"));
102
103     Vehiculo clon = original.clonar();
104
105     clon.setPlaca(dto.getPlaca());
106
107     if (dto.getLatitud() != null) {
108         clon.setLatitud(dto.getLatitud());
109     }
110
111     if (dto.getLongitud() != null) {
112         clon.setLongitud(dto.getLongitud());
113     }
114
115     if (dto.getEstado() != null) {
116         clon.setEstado(dto.getEstado());
117     }
118
119 }

```

En la imagen se observa la implementación del método `clonarVehiculo`, el cual permite aplicar el patrón Prototype dentro del sistema. Este método se encarga de generar una copia de un objeto existente, replicando sus atributos para crear una nueva instancia basada en el prototipo.

A través de este método, el sistema puede reutilizar configuraciones previamente definidas y crear nuevos objetos de forma más eficiente, evitando la instanciación manual completa y reduciendo la complejidad en la creación de entidades del dominio.

Además, el uso de este método garantiza que el objeto clonado sea una instancia independiente del original, permitiendo modificar sus atributos sin afectar el estado del prototipo inicial. Esto facilita la generación de múltiples objetos con características similares, manteniendo consistencia en la información y mejorando la flexibilidad del sistema ante futuros cambios o ampliaciones.



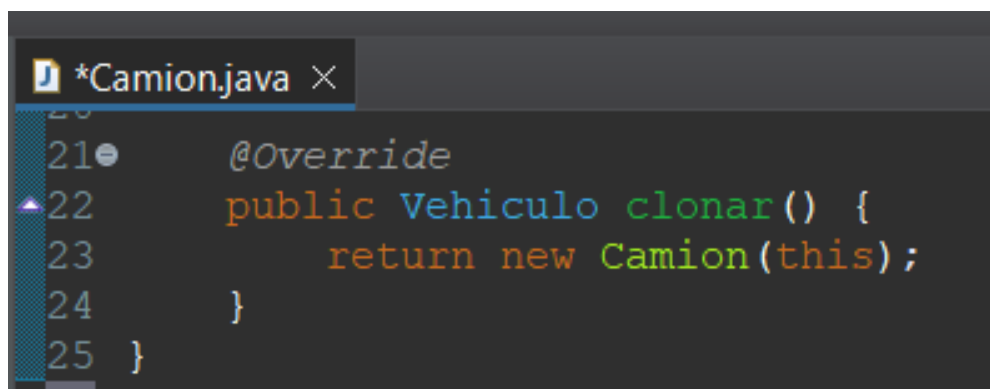
```

60
61 // =====
62 // ENDPOINT PROTOTYPE
63 // =====
64
65 @PostMapping("/{placaOriginal}/clonar")
66 public Vehiculo clonarVehiculo(@PathVariable String placaOriginal,
67                               @RequestBody VehiculoDTO dto) {
68     return vehiculoService.clonarVehiculo(placaOriginal, dto);
69 }
70 }

```

En la imagen se observa la definición del endpoint mediante la anotación `@PostMapping`, el cual permite recibir solicitudes externas para la creación de nuevos objetos dentro del sistema. En este caso, el controlador actúa como punto de entrada para invocar la lógica de clonación del vehículo.

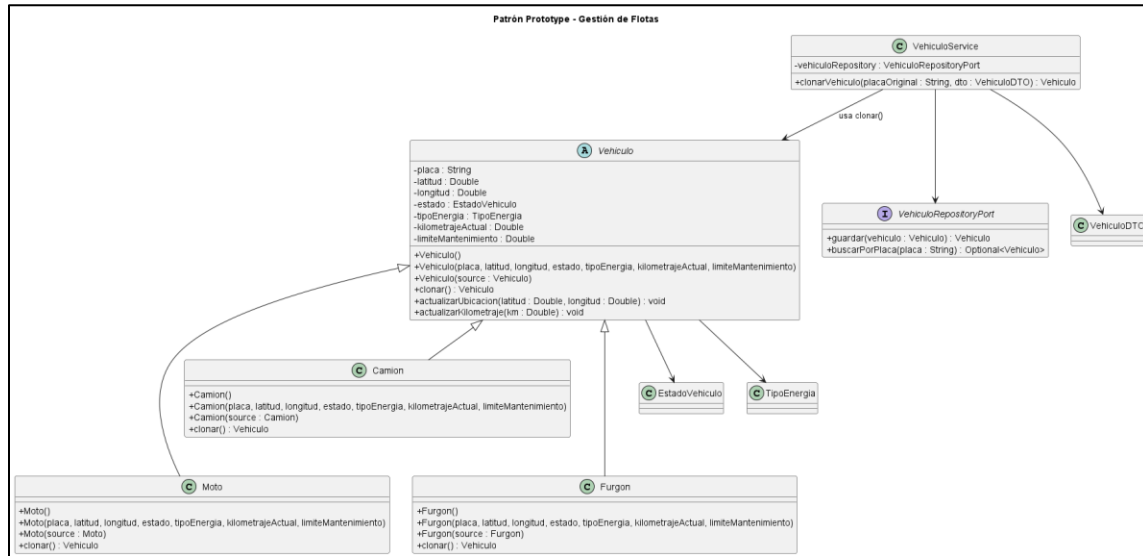
A través de este endpoint, el sistema recibe la información necesaria y delega la operación al servicio correspondiente, donde se aplica el patrón Prototype para generar una nueva instancia basada en un objeto existente. Esto demuestra cómo el patrón se integra con la arquitectura del sistema y es utilizado en escenarios reales de operación.



```
*Camion.java x
21 @Override
22 public Vehiculo clonar() {
23     return new Camion(this);
24 }
25 }
```

El uso de `@Override` indica que el método está definido en una interfaz o clase base, lo que permite mantener una estructura organizada y desacoplada. En este punto se aplica directamente el patrón Prototype, asegurando que la creación del objeto se realice mediante clonación y no por instanciación directa.

DIAGRAMA UML



PRUEBAS

POST `http://localhost:8080/vehiculos/CAMION`

Params Authorization Headers (8) Body Pre-reqs

none form-data x-www-form-urlencoded raw

```
1 {
2   "placa": "REC123",
3   "latitud": 10.45,
4   "longitud": -73.25,
5   "estado": "DISPONIBLE",
6   "tipoEnergia": "GASOLINA",
7   "kilometrajeActual": 50000,
8   "limiteMantenimiento": 80000
9 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "estado": "DISPONIBLE",
3   "kilometrajeActual": 50000.0,
4   "latitud": 10.45,
5   "limiteMantenimiento": 80000.0,
6   "longitud": -73.25,
7   "placa": "REC123",
8   "tipoEnergia": "GASOLINA"
9 }
```

HTTP `http://localhost:8080/vehiculos/REC123/clonar`

POST `http://localhost:8080/vehiculos/REC123/clonar`

Params Authorization Headers (8) Body Pre-request Script

none form-data x-www-form-urlencoded raw binary

```
1 {
2   "placa": "XYZ999",
3   "latitud": 11.20,
4   "longitud": -74.10,
5   "estado": "DISPONIBLE",
6   "tipoEnergia": "ELECTRICO",
7   "kilometrajeActual": 1000,
8   "limiteMantenimiento": 80000
9 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "estado": "DISPONIBLE",
3   "kilometrajeActual": 1000.0,
4   "latitud": 11.2,
5   "limiteMantenimiento": 80000.0,
6   "longitud": -74.1,
7   "placa": "XYZ999",
8   "tipoEnergia": "ELECTRICO"
9 }
```

Para validar la correcta implementación del patrón Prototype dentro del sistema de gestión de flotas, se realizaron pruebas mediante la herramienta Postman, utilizando el endpoint expuesto para la clonación de vehículos.

En las pruebas se enviaron solicitudes HTTP de tipo POST hacia el controlador del sistema, donde se recibe la información necesaria para identificar el objeto base (prototipo) que será clonado. A partir de esta solicitud, el sistema invoca la lógica del servicio, el cual ejecuta el método `clonarVehiculo` para generar una nueva instancia basada en el objeto original.

Los resultados obtenidos en las respuestas del sistema evidencian que se crea correctamente un nuevo vehículo con atributos similares al prototipo, manteniendo coherencia en la información. Adicionalmente, se puede observar que el objeto generado es independiente del original, permitiendo la modificación de sus atributos sin afectar la instancia base.

Estas pruebas confirman que el sistema implementa correctamente el patrón Prototype, ya que la creación de objetos se realiza mediante clonación en lugar de instanciación directa, optimizando el proceso y reduciendo la complejidad en la construcción de entidades del dominio.

PATRON ADAPTER

El patrón Adapter es un patrón de diseño estructural cuyo propósito es permitir la comunicación entre clases o sistemas con interfaces incompatibles, actuando como un intermediario que traduce una interfaz en otra que el cliente puede utilizar.

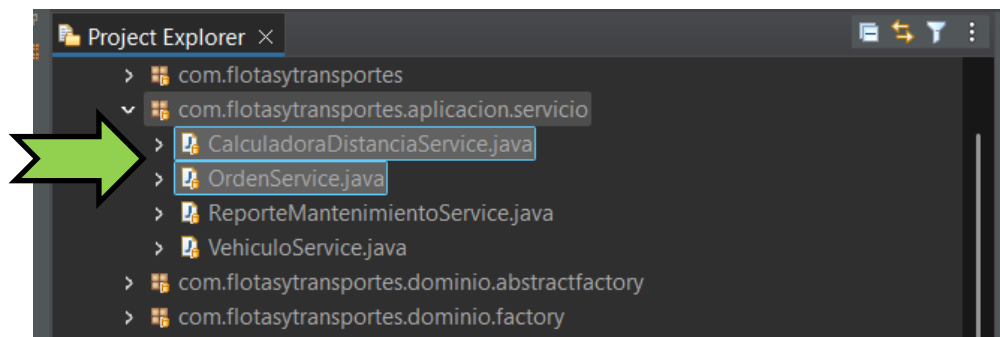
Este patrón resulta fundamental cuando se desea integrar componentes existentes o sistemas externos sin modificar su código original, ya que encapsula la lógica de conversión necesaria para que ambos puedan interactuar de manera transparente. De esta forma, el Adapter evita cambios directos en las clases del dominio, promoviendo el principio de bajo acoplamiento y facilitando la extensibilidad del sistema.

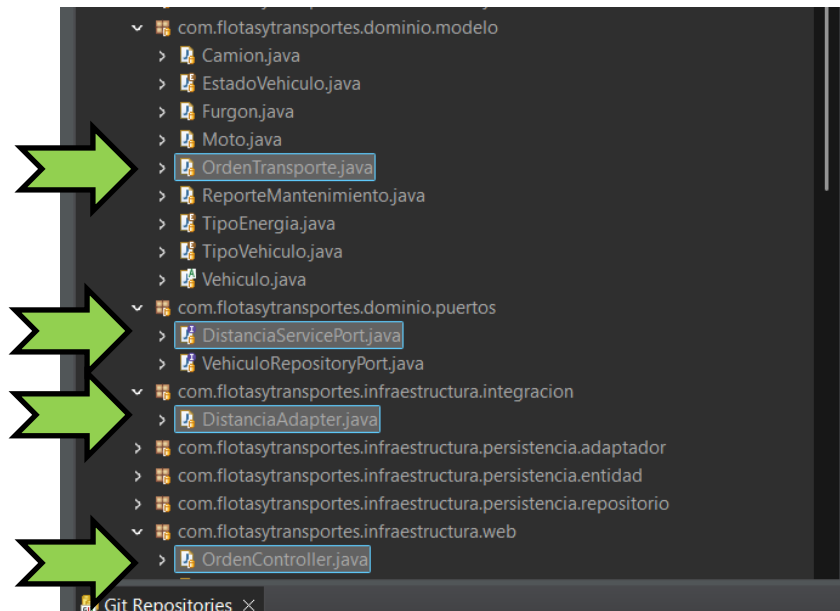
En el sistema de gestión de flotas, el patrón Adapter se implementa dentro de la arquitectura hexagonal, específicamente en la capa de infraestructura. Aquí, los adapters permiten conectar los puertos definidos en el dominio con las implementaciones concretas, como repositorios basados en JPA o servicios externos.

Este enfoque permite que la lógica de negocio permanezca completamente independiente de detalles técnicos, facilitando la sustitución de tecnologías (por ejemplo, cambiar de base de datos o framework) sin afectar el núcleo del sistema.

IMPLEMENTACION EN EL CODIGO

El patrón Adapter se implementa en la clase:





En la imagen se observan las clases involucradas en la implementación del patrón Adapter, junto con sus relaciones representadas mediante flechas. Se identifica la interfaz del dominio, la clase DistanceAdapter y la clase concreta encargada del cálculo de distancias.

Las flechas indican la relación de implementación y dependencia entre las clases, evidenciando cómo el DistanceAdapter actúa como intermediario al implementar la interfaz esperada y adaptar la funcionalidad de la clase existente.

Esta estructura permite que el sistema utilice el cálculo de distancias sin depender directamente de la implementación concreta, garantizando un bajo acoplamiento y facilitando la integración de nuevas funcionalidades sin afectar la lógica principal.

The image shows the code for the `DistanciaAdapter` class in the `OrdenTransporte.java` file. The code is as follows:

```
1 package com.flotasytransportes.src/main/java/com/flotasytransportes/dominio/modelo/OrdenTransporte.java
2
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class DistanciaAdapter implements DistanciaServicePort {
7
8     private final CalculadoraDistanciaService calculadoraDistanciaService;
9
10    public DistanciaAdapter(CalculadoraDistanciaService calculadoraDistanciaService) {
11        this.calculadoraDistanciaService = calculadoraDistanciaService;
12    }
13
14    @Override
15    public double calcularDistancia(double lat1, double lon1, double lat2, double lon2) {
16        return calculadoraDistanciaService.calcularDistancia(lat1, lon1, lat2, lon2);
17    }
18 }
19
20
21
22 }
```

A green arrow points to the line `public class DistanciaAdapter implements DistanciaServicePort`.

En la imagen se observa la clase DistanceAdapter, la cual implementa el patrón Adapter dentro del sistema. Esta clase actúa como un intermediario encargado de adaptar el cálculo de distancias a la interfaz que el sistema espera utilizar.

El DistanceAdapter permite integrar una lógica de cálculo externa o diferente (por ejemplo, otra forma de medir distancias) sin modificar la estructura del sistema, traduciendo los datos y operaciones hacia el formato requerido por el dominio.

```
OrdenTransporte.java  DistanciaAdapter.java  DistanciaServicePort.java x  OrdenService.java  CalculadoraDist
```

```
1 package com.flotasytransportes.dominio.puertos;
2
3 public interface DistanciaServicePort {
4
5     double calcularDistancia(double lat1, double lon1, double lat2, double lon2);
6
7 }
```

Se observa la interfaz DistanciaServicePort, la cual representa el puerto definido en el dominio del sistema. Esta interfaz establece las operaciones necesarias para el cálculo de distancias, sin depender de una implementación específica.

Dentro del patrón Adapter, esta interfaz actúa como el contrato que debe ser implementado por el DistanceAdapter, permitiendo que el sistema utilice el servicio de cálculo de distancias de manera desacoplada.

```
OrdenTransporte.java  DistanciaAdapter.java  DistanciaServicePort.java  OrdenService.java x  CalculadoraDistanciaSe
```

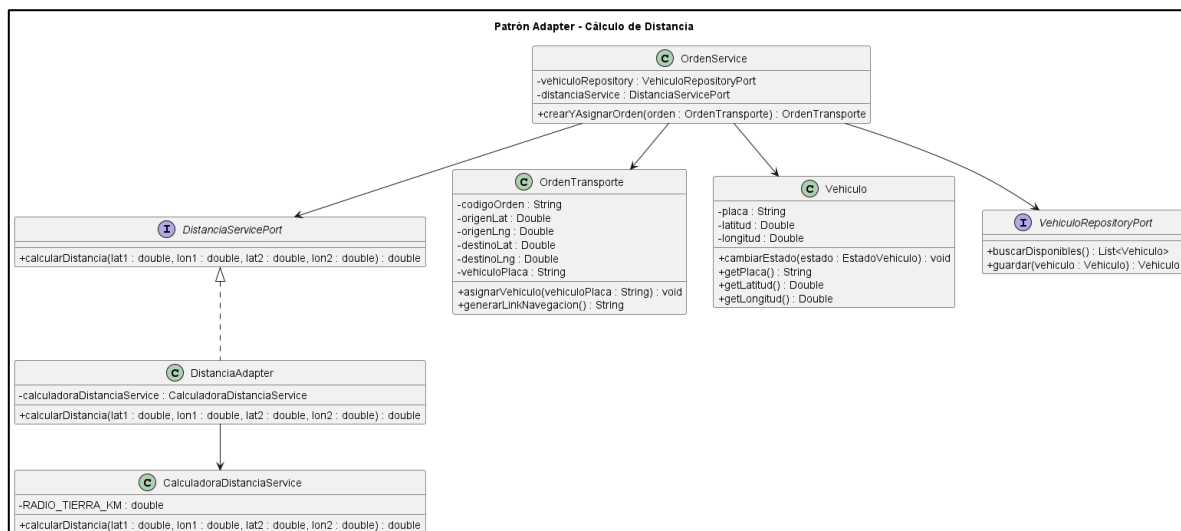
```
1 package com.flotasytransportes.aplicacion.servicio;
2
3 import com.flotasytransportes.dominio.modelo.*;
4
5 @Service
6 public class OrdenService {
7
8     private final VehiculoRepositoryPort vehiculoRepository;
9     private final DistanciaServicePort distanciaService;
10
11     public OrdenService(VehiculoRepositoryPort vehiculoRepository,
12                         DistanciaServicePort distanciaService) {
13         this.vehiculoRepository = vehiculoRepository;
14         this.distanciaService = distanciaService;
15     }
16
17     public OrdenTransporte crearYAsignarOrden(OrdenTransporte orden) {
18
19         List<Vehiculo> disponibles = vehiculoRepository.buscarDisponibles();
20
21         if (disponibles.isEmpty()) {
22             throw new RuntimeException("No hay vehiculos disponibles");
23         }
24
25         Vehiculo vehiculoMasCercano = disponibles.stream()
26             .min(Comparator.comparing(v -> distanciaService.calcularDistancia(
27                 orden.getOrigenLat(),
28                 orden.getOrigenLng(),
29                 v.getLatitude(),
30                 v.getLongitude()
31             )))
32             .get();
33     }
34 }
```

La clase OrdenService, la cual forma parte de la capa de aplicación y se encarga de gestionar la lógica relacionada con las órdenes dentro del sistema.

Esta clase utiliza la interfaz DistanciaServicePort para realizar el cálculo de distancias, sin depender directamente de una implementación concreta. De esta manera, el servicio delega esta responsabilidad al Adapter, cumpliendo con el principio de bajo acoplamiento.

```
1 package com.flotasytransportes.infraestructura.web;
2
3 import com.flotasytransportes.aplicacion.servicioOrdenService;
4
5 @RestController
6 @RequestMapping("/ordenes")
7 public class OrdenController {
8
9     private final OrdenService ordenService;
10
11     public OrdenController(OrdenService ordenService) {
12         this.ordenService = ordenService;
13     }
14
15     @PostMapping("/asignar")
16     public OrdenTransporte crearYAsignar(@RequestBody OrdenTransporte orden) {
17         return ordenService.crearYAsignarOrden(orden);
18     }
19 }
20
21
22
```

DIAGRAMA UML



PRUEBAS

```
===== DEMOSTRACIÓN ADAPTER =====
[CLIENTE - OrdenService] Creando orden: ORD010
[CLIENTE] Origen: (7.125, -73.119)
[CLIENTE] Buscando vehiculos disponibles...
Hibernate: select ve1_0.placa,ve1_0.estado,ve1_0.kilometraje_actual,ve1_0.latitud,ve1_0.limite_mantenimiento,ve1_0.longitud,ve1_0.tipo,ve1_0.tipo_energia from vehiculos ve1_0 where ve1_0.estado=?
[ADAPTER - DistanciaAdapter] Adaptando solicitud de cálculo...
[ADAPTEE - CalculadoraDistanciaService] Calculando distancia...
  Punto A: (7.125, -73.119)
  Punto B: (11.2, -74.1)
[ADAPTEE] Distancia calculada: 465.734978409879 km
[CLIENTE] Vehículo GH999 está a 465.734978409879 km
[CLIENTE] Vehículo más cercano seleccionado: GH999
Hibernate: update vehiculos set estado=?,kilometraje_actual=?,latitud=?,limite_mantenimiento=?,longitud=?,tipo=?,tipo_energia=? where placa=?
[CLIENTE] Orden asignada al vehículo: GH999
[CLIENTE] Link de navegación: https://www.google.com/maps/dir/7.125,-73.119/7.2,-73.05
=====
```

En la imagen se evidencia la ejecución completa del sistema durante una prueba del patrón Adapter, mostrando los mensajes generados en consola mediante `System.out.println`.

Inicialmente, el sistema registra la creación de una orden desde la clase `OrdenService`, indicando el origen y la búsqueda de vehículos disponibles. Posteriormente, se observa cómo el flujo pasa al `DistanceAdapter`, el cual actúa como intermediario y delega el cálculo de distancia a una implementación específica (`CalculadoraDistanciaService`).

Durante la ejecución, se muestra el cálculo de la distancia entre dos puntos geográficos, evidenciando que el Adapter traduce correctamente la solicitud hacia el servicio externo o adaptado. Finalmente, el sistema selecciona el vehículo más cercano y asigna la orden, confirmando que toda la operación se realizó correctamente.

PATRON BRIDGE

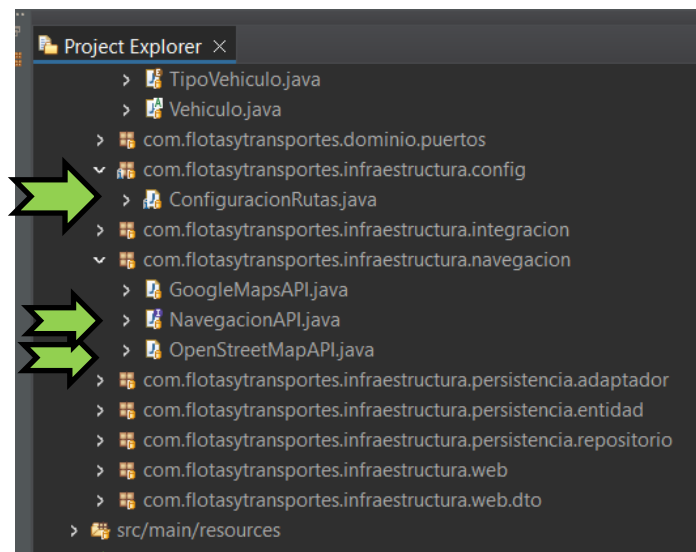
El patrón Bridge es un patrón de diseño estructural cuyo objetivo principal es separar una abstracción de su implementación, permitiendo que ambas puedan evolucionar de manera independiente sin generar dependencias directas entre ellas.

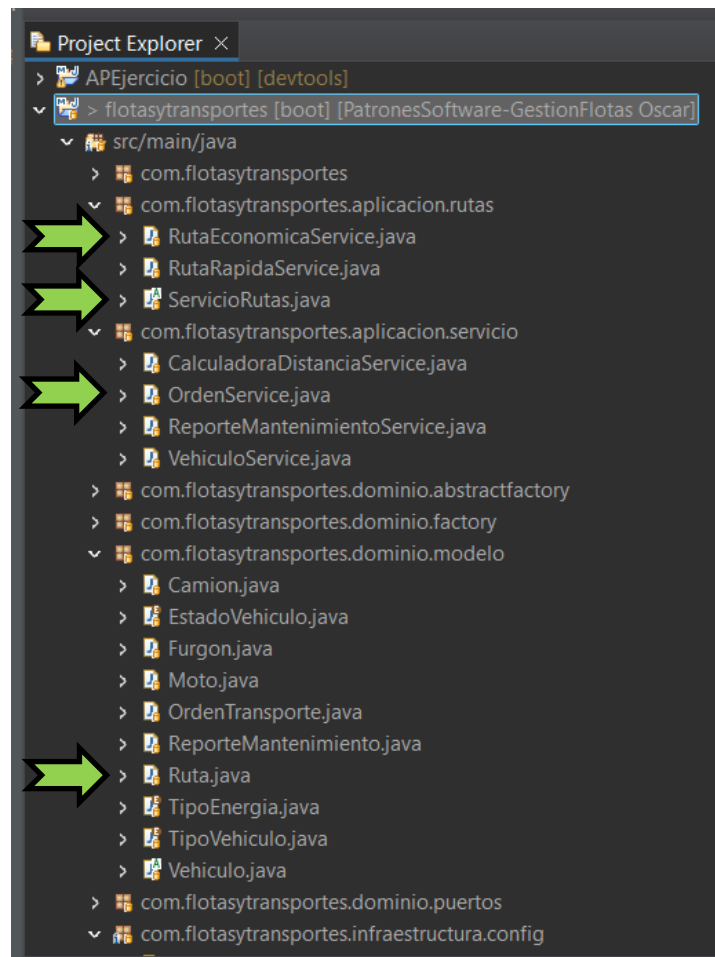
Este patrón resulta especialmente útil en escenarios donde una clase puede tener múltiples variaciones tanto en su estructura como en su comportamiento. En lugar de crear una gran cantidad de clases combinadas para cubrir todas las posibles variaciones, el patrón Bridge propone dividir el sistema en dos jerarquías independientes: por un lado, la abstracción, que define la lógica principal del sistema, y por otro, la implementación, que contiene los detalles específicos de funcionamiento.

En el entorno del sistema de gestión de flotas, el patrón Bridge permite desacoplar la lógica principal de ciertos procesos del sistema de las diferentes formas en que estos pueden ser implementados. Por ejemplo, una misma funcionalidad puede tener distintas maneras de ejecutarse dependiendo del tipo de servicio o tecnología utilizada, y gracias al uso del Bridge, estas variaciones pueden manejarse sin necesidad de modificar la estructura principal del sistema.

IMPLEMENTACION EN EL CODIGO

El patrón Bridge se implementa en la clase:





En las anteriores imágenes se presenta la definición de la interfaz que forma parte de la implementación del patrón Bridge dentro del sistema. Esta interfaz actúa como un contrato que define las operaciones que deben ser implementadas por las clases concretas, permitiendo separar la lógica principal del sistema de los detalles específicos de ejecución.

Mediante esta estructura, la abstracción puede delegar el comportamiento a diferentes implementaciones sin establecer una dependencia directa con ellas, lo que favorece un bajo acoplamiento y permite que el sistema sea más flexible y fácilmente extensible ante futuros cambios.

```

1 package com.flotasytransportes.infraestructura.config;
2
3 import org.springframework.beans.factory.annotation.Qualifier;
10
11 @Configuration
12 public class ConfiguracionRutas {
13
14     @Bean
15     public ServicioRutas servicioRutas(
16         @Qualifier("googleMapsAPI") NavegacionAPI api) {
17
18         return new RutaRapidaService(api);
19     }
20 }
21

```

En esta estructura, la interfaz NavegacionAPI representa la implementación (Implementor) del patrón, ya que define las operaciones relacionadas con la navegación y el cálculo de rutas. Por su parte, la clase RutaRapidaService, que implementa la interfaz ServicioRutas, actúa como la abstracción (Abstraction), encargándose de la lógica del sistema sin depender de una implementación concreta.

El patrón Bridge se materializa en el momento en que la clase RutaRapidaService recibe como parámetro una instancia de NavegacionAPI, permitiendo delegar en ella las operaciones específicas. Esta relación se configura mediante la anotación @Bean, donde se inyecta una implementación concreta (por ejemplo, googleMapsAPI) sin acoplar directamente la lógica del servicio a dicha implementación.

```

1 package com.flotasytransportes.aplicacion.rutas;
2
3 import com.flotasytransportes.infraestructura.navegacion.NavegacionAPI;
5
6 public class RutaEconomicaService extends ServicioRutas {
7
8     public RutaEconomicaService(NavegacionAPI api) {
9         super(api);
10     }
11
12     @Override
13     public Ruta generarRuta(String origen, String destino) {
14         System.out.println("Ruta económica");
15         return navegacionAPI.calcularRuta(origen, destino);
16     }
17 }
18

```



```

1 package com.flotasytransportes.aplicacion.rutas;
2
3 import com.flotasytransportes.infraestructura.navegacion.NavegacionAPI;
4
5
6 public class RutaRapidaService extends ServicioRutas {
7
8     public RutaRapidaService(NavegacionAPI api) {
9         super(api);
10    }
11
12    @Override
13    public Ruta generarRuta(String origen, String destino) {
14        System.out.println("Ruta rápida");
15        return navegacionAPI.calcularRuta(origen, destino);
16    }
17 }
18

```

En este caso, la clase define un comportamiento específico para la generación de rutas, correspondiente a una estrategia de tipo económica. A través del método sobrescrito generarRuta, se delega el cálculo de la ruta a la implementación de NavegacionAPI, manteniendo desacoplada la lógica de negocio de los detalles concretos de la API de navegación.

El patrón Bridge se evidencia en la forma en que la clase recibe la implementación mediante el constructor y la utiliza a través de la composición heredada, sin depender de una implementación específica. Esto permite que la lógica de generación de rutas pueda variar (por ejemplo, rápida o económica) sin afectar la forma en que se realiza el cálculo, el cual queda delegado a la interfaz de navegación.

```

1 package com.flotasytransportes.dominio.modelo;
2
3 public class Ruta {
4     private String origen;
5     private String destino;
6     private double distancia;
7
8     public Ruta(String origen, String destino, double distancia) {
9         this.origen = origen;
10        this.destino = destino;
11        this.distancia = distancia;
12    }
13
14    public String getOrigen() {
15        return origen;
16    }
17
18    public void setOrigen(String origen) {
19        this.origen = origen;
20    }
21

```


Esta clase actúa como un objeto de transferencia de datos dentro del sistema, siendo utilizada como resultado del proceso de cálculo de rutas. En el contexto del patrón Bridge, la clase Ruta es el objeto que se genera a partir de la interacción entre la abstracción (ServicioRutas y sus variantes) y la implementación (NavegacionAPI).

```

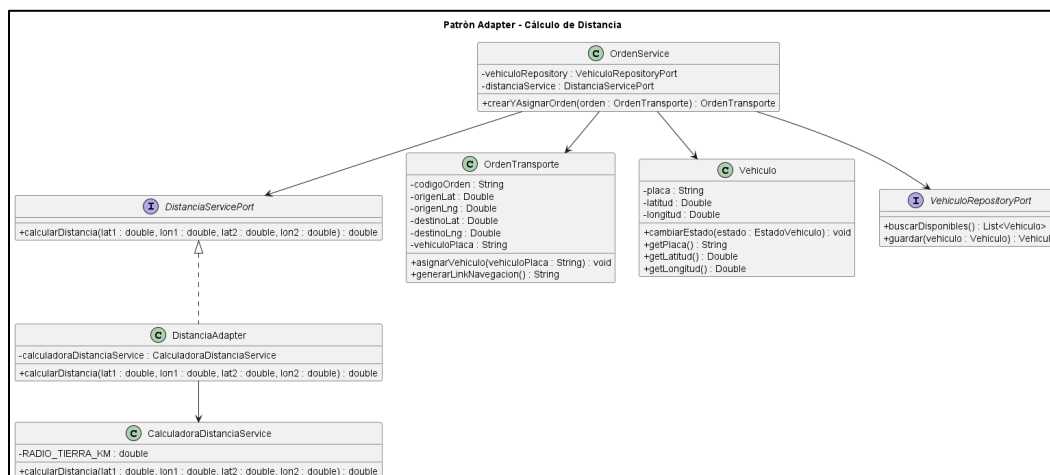
1 package com.flotasytransportes.infraestructura.navegacion;
2
3 import com.flotasytransportes.dominio.modelo.*;
4
5 public interface NavegacionAPI {
6     Ruta calcularRuta(String origen, String destino);
7     double calcularDistancia(String origen, String destino);
8 }
9

```

En el contexto del sistema, esta interfaz permite desacoplar la lógica de generación de rutas de los detalles específicos de cómo se realiza el cálculo, ya sea mediante APIs externas o diferentes algoritmos. De esta forma, las clases que pertenecen a la abstracción, como ServicioRutas y sus derivadas, interactúan únicamente con esta interfaz sin conocer la implementación concreta.

El patrón Bridge se evidencia en que NavegacionAPI actúa como punto de conexión entre la abstracción y las implementaciones concretas, permitiendo que el sistema pueda integrar diferentes proveedores de navegación (por ejemplo, Google Maps u otros servicios) sin modificar la lógica principal.

DIAGRAMA UML



PRUEBAS

```
[CLIENTE] Generando ruta con Bridge...
Ruta rápida
Google Maps en uso
[CLIENTE] Distancia de ruta: 10.0
Hibernate: update vehiculos set estado=?,kilometraje_actual=?,latitud=?,limite_mantenimiento=?,longitud=?,tipo=?,tipo_energia=? where placa=?
[CLIENTE] Orden asignada correctamente
=====
```

En la imagen se evidencia la ejecución de una prueba del sistema donde se aplica el patrón Bridge en el proceso de generación de rutas. Inicialmente, el sistema indica que se está generando una ruta utilizando la estructura del patrón, mostrando el mensaje “Generando ruta con Bridge...”.

Posteriormente, se selecciona el tipo de abstracción correspondiente, en este caso Ruta rápida, lo que indica que se está utilizando una implementación específica de la abstracción (como RutaRapidaService).

A continuación, se observa el mensaje “Google Maps en uso”, lo que confirma que el sistema está utilizando una implementación concreta de la interfaz NavegacionAPI, evidenciando cómo la abstracción delega la ejecución en la implementación sin acoplarse directamente a ella.

El sistema continúa mostrando el resultado del cálculo, como la distancia de la ruta, y posteriormente ejecuta operaciones adicionales como la actualización del vehículo en la base de datos y la asignación de la orden.

PATRON DECORATOR

El patrón Decorator es un patrón de diseño estructural que permite agregar funcionalidades adicionales a un objeto de manera dinámica, sin necesidad de modificar su estructura original. Este patrón se fundamenta en el principio de composición sobre herencia, ya que, en lugar de crear múltiples subclases para extender comportamientos, se utilizan objetos decoradores que envuelven al objeto base.

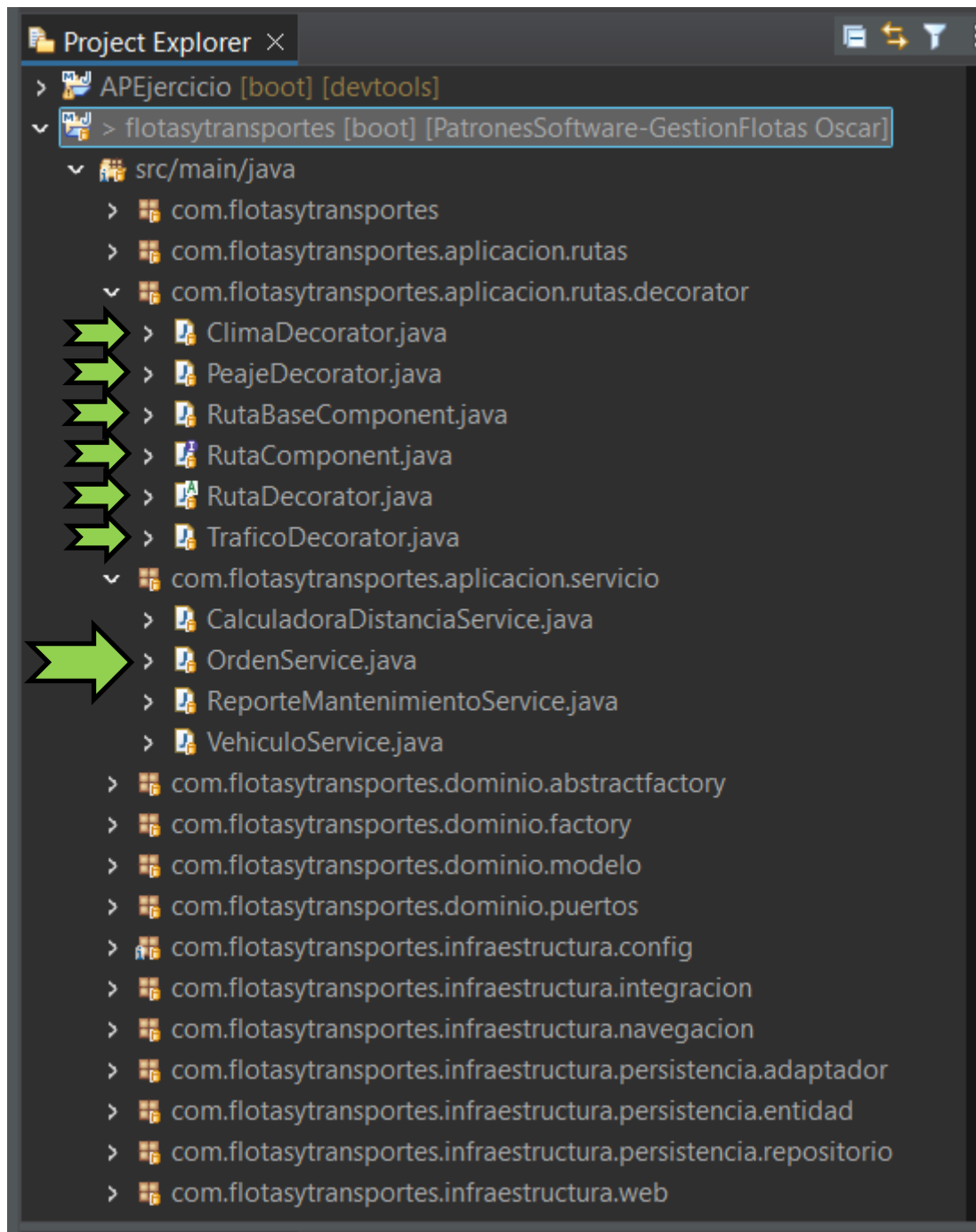
El funcionamiento del patrón consiste en crear una jerarquía de clases donde tanto el componente base como los decoradores comparten una misma interfaz. De esta forma, los decoradores pueden sustituir al objeto original en cualquier parte del sistema, manteniendo la compatibilidad y permitiendo la extensión del comportamiento de manera transparente.

A diferencia de otros enfoques, el patrón Decorator permite añadir responsabilidades de forma incremental, es decir, se pueden combinar múltiples decoradores sobre un mismo objeto, logrando así una gran flexibilidad en la construcción de funcionalidades sin generar una explosión de clases en el sistema.

En el contexto del sistema de gestión de flotas, el patrón Decorator se utiliza para extender funcionalidades sobre componentes existentes, como la generación de reportes, el procesamiento de información o la adición de características adicionales a ciertos servicios, sin modificar su implementación base.

IMPLEMENTACION EN EL CODIGO

El patrón Decorator se implementa en la clase:



En la imagen se observa la estructura del proyecto dentro del entorno de desarrollo, donde se identifica el paquete correspondiente a la implementación del patrón Decorator. En este se

agrupan las diferentes clases que participan en la extensión de funcionalidades relacionadas con la gestión de rutas.

Se puede apreciar la organización de las clases en torno a un componente base, una implementación principal y varios decoradores que permiten añadir nuevas características de manera progresiva. Esta estructura facilita la extensión del comportamiento del sistema sin modificar las clases originales.

```
ClimaDecorator.java X
1 package com.flotasytransportes.aplicacion.rutas.decorator;
2
3 import com.flotasytransportes.dominio.modelo.Ruta;
4
5 public class ClimaDecorator extends RutaDecorator {
6
7     public ClimaDecorator(RutaComponent wrappee) {
8         super(wrappee);
9     }
10
11     @Override
12     public Ruta procesarRuta(Ruta ruta) {
13         ruta = super.procesarRuta(ruta);
14
15         System.out.println("[DECORATOR] Ajustando por condiciones climáticas");
16         return new Ruta(ruta.getOrigen(), ruta.getDestino(), ruta.getDistancia() + 1.5);
17     }
18 }
```

A través del método sobrescrito `procesarRuta`, el decorador primero delega la ejecución al objeto base mediante la llamada a `super.procesarRuta(ruta)`, lo que garantiza que se mantenga el comportamiento original. Posteriormente, se añade una funcionalidad adicional, en este caso, el ajuste de la distancia de la ruta en función de las condiciones climáticas.

Este comportamiento se refleja en la modificación del atributo `distancia`, incrementándolo en un valor específico, lo que simula el impacto del clima sobre la ruta. De esta manera, el decorador extiende la funcionalidad del objeto sin alterar su implementación base.

```

1 package com.flotasytransportes.aplicacion.rutas.decorator;
2
3 import com.flotasytransportes.dominio.modelo.Ruta;
4
5 public class PeajeDecorator extends RutaDecorator {
6
7     public PeajeDecorator(RutaComponent wrappee) {
8         super(wrappee);
9     }
10
11     @Override
12     public Ruta procesarRuta(Ruta ruta) {
13         ruta = super.procesarRuta(ruta);
14
15         System.out.println("[DECORATOR] Añadiendo costo de peajes");
16         return new Ruta(ruta.getOrigen(), ruta.getDestino(), ruta.getDistancia() + 1);
17     }
18 }

```

En la imagen se observa la clase PeajeDecorator, la cual corresponde a un decorador concreto dentro del patrón Decorator. Esta clase extiende de RutaDecorator, lo que le permite envolver un objeto del tipo RutaComponent y añadir nuevas funcionalidades de manera dinámica sin modificar la implementación base.

En el método procesarRuta, el decorador inicia delegando la ejecución al componente envuelto mediante la llamada a super.procesarRuta(ruta), asegurando que se conserve el comportamiento original del sistema. Posteriormente, se incorpora una funcionalidad adicional relacionada con el cálculo de costos de peajes.

Este comportamiento se refleja en la modificación del atributo distancia de la ruta, incrementándolo en un valor específico, lo que simula el impacto de los peajes dentro del recorrido. De esta forma, el decorador añade una nueva responsabilidad sin alterar la lógica del componente base.

```

1 package com.flotasytransportes.aplicacion.rutas.decorator;
2
3 import com.flotasytransportes.dominio.modelo.Ruta;
4
5 public class RutaBaseComponent implements RutaComponent {
6
7     @Override
8     public Ruta procesarRuta(Ruta ruta) {
9         System.out.println("[BASE] Ruta sin modificaciones");
10        return ruta;
11    }
12 }

```

A través del método `procesarRuta`, se ejecuta la funcionalidad original, retornando la ruta sin alteraciones y mostrando un mensaje que indica que no se han aplicado cambios. Esta clase representa el punto de partida sobre el cual se construirán las extensiones mediante los diferentes decoradores.

En el contexto del patrón Decorator, `RutaBaseComponent` es el objeto que será envuelto por los decoradores concretos, permitiendo que se le añadan nuevas funcionalidades de forma progresiva sin modificar su implementación original.

Esta estructura garantiza el cumplimiento del principio de abierto/cerrado, ya que el comportamiento del sistema puede ser extendido mediante decoradores sin necesidad de alterar la clase base, favoreciendo la flexibilidad y mantenibilidad del software.

```
RutaComponent.java ×
1 package com.flotasytransportes.aplicacion.rutas.decorator;
2
3 import com.flotasytransportes.dominio.modelo.Ruta;
4
5 public interface RutaComponent {
6     Ruta procesarRuta(Ruta ruta);
7 }
```

En la imagen se observa la interfaz `RutaComponent`, la cual representa el componente base (Component) dentro de la implementación del patrón Decorator. Esta interfaz define el contrato común que deben cumplir tanto el componente concreto como los decoradores.

A través del método `procesarRuta`, se establece la operación principal del sistema relacionada con el procesamiento de rutas, la cual podrá ser extendida por los diferentes decoradores sin modificar su definición original.

Esta estructura permite que todos los objetos involucrados en el patrón compartan la misma interfaz, lo que garantiza que los decoradores puedan sustituir al componente base de manera transparente dentro del sistema.

```

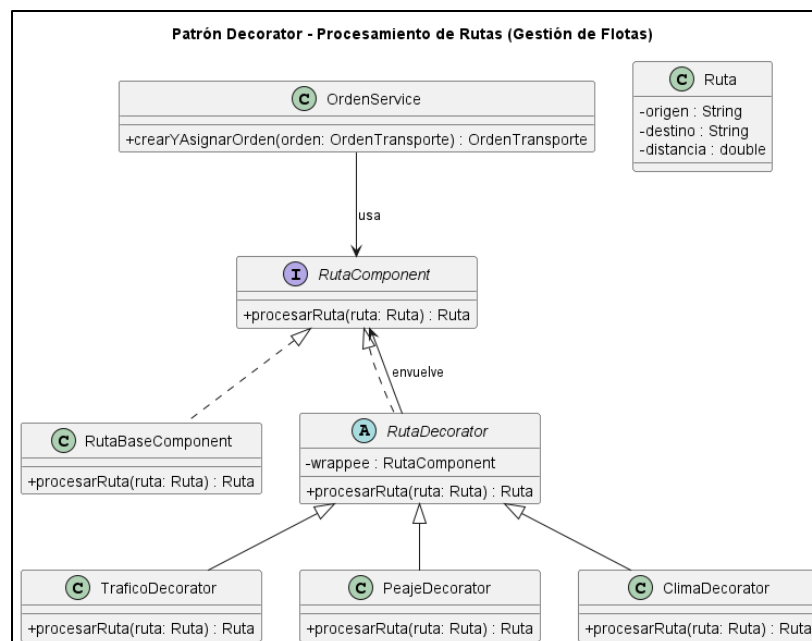
1 package com.flotasytransportes.aplicacion.rutas.decorator;
2
3 import com.flotasytransportes.dominio.modelo.Ruta;
4
5 public class TraficoDecorator extends RutaDecorator {
6
7     public TraficoDecorator(RutaComponent wrappee) {
8         super(wrappee);
9     }
10
11     @Override
12     public Ruta procesarRuta(Ruta ruta) {
13         ruta = super.procesarRuta(ruta);
14
15         System.out.println("[DECORATOR] Ajustando ruta por tráfico");
16         return new Ruta(ruta.getOrigen(), ruta.getDestino(), ruta.getDistancia() + 2);
17     }
18 }

```

En el método `procesarRuta`, el decorador inicia delegando la ejecución al componente envuelto mediante la llamada a `super.procesarRuta(ruta)`, garantizando que se mantenga el comportamiento base previamente definido. Posteriormente, se incorpora una funcionalidad adicional relacionada con el ajuste de la ruta en función de las condiciones de tráfico.

Este comportamiento se refleja en el incremento del atributo `distancia` de la ruta, simulando el impacto del tráfico en el recorrido. De esta manera, el decorador añade una nueva responsabilidad al objeto sin alterar su implementación original.

DIAGRAMA UML



PRUEBAS

```
[CLIENTE] Aplicando decoradores a la ruta...
[BASE] Ruta sin modificaciones
[DECORATOR] Ajustando ruta por tráfico
[DECORATOR] Añadiendo costo de peajes
[DECORATOR] Ajustando por condiciones climáticas
[CLIENTE] Ruta final con decoradores: 14.5
Hibernate: update vehiculos set estado=?,kilometraje_actual=?,latitud=?,limite_mantenimiento=?,longitud=?,tipo=?,tipo_energia=? where placa=?
[CLIENTE] Orden asignada correctamente
=====
```

En la imagen se evidencia la ejecución de una prueba del sistema donde se aplica el patrón Decorator para el procesamiento de rutas. Inicialmente, el sistema indica que se están aplicando decoradores sobre la ruta, lo que marca el inicio del encadenamiento de funcionalidades adicionales.

En primer lugar, se ejecuta el componente base, representado por el mensaje “Ruta sin modificaciones”, lo que confirma que el objeto inicial corresponde a la implementación base (RutaBaseComponent).

Posteriormente, se observa la aplicación secuencial de los diferentes decoradores: primero el ajuste por tráfico, luego la adición de costos de peajes y finalmente la modificación por condiciones climáticas. Cada uno de estos pasos corresponde a un decorador concreto (TráficoDecorator, PeajeDecorator y ClimaDecorator), los cuales envuelven el objeto base y añaden nuevas responsabilidades de forma progresiva.

El resultado final muestra una distancia total modificada (“Ruta final con decoradores: 14.5”), lo que evidencia cómo cada decorador aporta un cambio incremental al valor original, demostrando el funcionamiento acumulativo del patrón.

CONCLUSIONES

El desarrollo del Sistema de Gestión de Flotas permitió aplicar de manera práctica diversos conceptos fundamentales de la ingeniería de software orientada a objetos, demostrando cómo una adecuada organización arquitectónica puede contribuir a la construcción de sistemas más mantenibles, escalables y desacoplados. A través de la implementación de la arquitectura hexagonal, fue posible separar claramente la lógica del dominio de las dependencias tecnológicas externas, garantizando que las reglas del negocio permanezcan independientes de frameworks, bases de datos o interfaces externas.

La utilización de esta arquitectura permitió estructurar el sistema en diferentes capas bien definidas, como dominio, aplicación e infraestructura, facilitando la comprensión del flujo de funcionamiento del sistema y promoviendo el principio de inversión de dependencias. Gracias a este enfoque, el núcleo del sistema se mantiene protegido de cambios tecnológicos, permitiendo que la aplicación pueda evolucionar o adaptarse a nuevas tecnologías sin afectar la lógica principal del negocio.

Asimismo, la implementación de distintos patrones de diseño creacionales, como Singleton, Factory Method, Abstract Factory y Builder, permitió mejorar significativamente la organización del código y la gestión de la creación de objetos dentro del sistema. El patrón Singleton fue utilizado mediante el contenedor de Spring para garantizar una única instancia de los servicios y repositorios, optimizando el uso de recursos y asegurando una gestión centralizada de la lógica del sistema.

Por su parte, el patrón Factory Method permitió delegar la creación de diferentes tipos de vehículos a fábricas especializadas, evitando el uso de estructuras condicionales complejas dentro de los servicios y reduciendo el acoplamiento entre componentes. De manera complementaria, el patrón Abstract Factory permitió organizar la creación de familias de objetos relacionados, garantizando que los distintos tipos de vehículos puedan ser generados de forma flexible y extensible dentro del sistema.

Mientras que, el patrón Builder fue utilizado para la construcción de reportes de mantenimiento, permitiendo crear objetos complejos de manera progresiva y estructurada. Este patrón facilitó la separación entre atributos obligatorios y opcionales, mejorando la

legibilidad del código y evitando la necesidad de utilizar constructores extensos con múltiples parámetros.

Adicionalmente, la implementación del patrón Prototype permitió optimizar la creación de objetos dentro del sistema mediante la clonación de instancias existentes. Esto facilitó la reutilización de configuraciones previamente definidas, reduciendo la complejidad en la instanciación de entidades y permitiendo generar nuevos objetos de manera eficiente sin afectar el estado del objeto original.

Por otra parte, el patrón Adapter permitió integrar componentes con interfaces diferentes, especialmente en la comunicación entre la lógica de negocio y servicios externos, como el cálculo de distancias. A través de este patrón, se logró desacoplar el dominio de las implementaciones concretas, garantizando que el sistema pueda interactuar con diferentes servicios sin depender directamente de ellos, lo que mejora la flexibilidad y mantenibilidad de la aplicación.

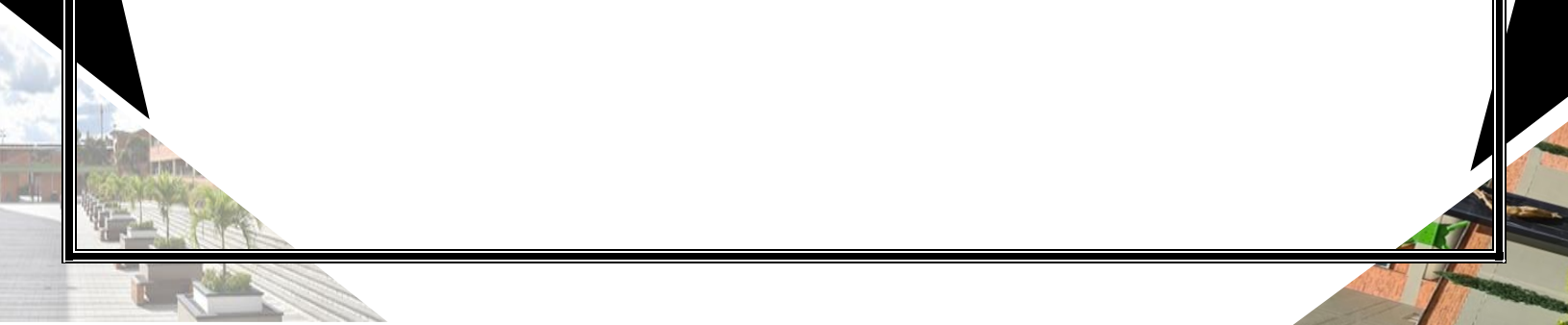
De igual manera, la implementación del patrón Bridge permitió separar la abstracción de la lógica de rutas de las implementaciones concretas de los servicios de navegación. Esto se evidenció en la forma en que el sistema delega el cálculo de rutas a diferentes APIs sin acoplarse directamente a ellas, permitiendo que tanto los tipos de rutas como las fuentes de navegación evolucionen de manera independiente, mejorando la flexibilidad y adaptabilidad del sistema.

Finalmente, el patrón Decorator permitió extender dinámicamente el comportamiento de las rutas mediante la adición de funcionalidades como condiciones climáticas, tráfico y costos de peajes. Este enfoque permitió combinar múltiples características sobre un mismo objeto sin modificar su implementación base, demostrando una alta flexibilidad y facilitando la escalabilidad del sistema.

Las pruebas realizadas mediante herramientas como Postman permitieron validar el correcto funcionamiento de los servicios implementados, confirmando que el sistema puede gestionar correctamente la creación, consulta, actualización y eliminación de vehículos, así como la generación de reportes de mantenimiento asociados a los mismos. Estas pruebas evidenciaron

que la arquitectura y los patrones de diseño aplicados cumplen adecuadamente su propósito dentro del sistema.

En conclusión, el desarrollo de este proyecto permitió demostrar cómo la aplicación adecuada de principios de diseño, arquitecturas desacopladas y patrones de diseño contribuye a la construcción de sistemas de software más robustos, organizados y fáciles de mantener. Este tipo de enfoques resulta fundamental en el desarrollo de aplicaciones modernas, especialmente en entornos donde los sistemas deben evolucionar constantemente para adaptarse a nuevas necesidades tecnológicas y de negocio.



REFERENCIAS BIBLIOGRAFICAS

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
<https://www.oreilly.com/library/view/effective-java-3rd/9780134686097/>

Cockburn, A. (2005). *Hexagonal architecture*. Recuperado de
<https://alistair.cockburn.us/hexagonal-architecture>

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.
<https://domainlanguage.com/ddd/>

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
<https://martinfowler.com/books/ea.html>

Freeman, E., Robson, E., Sierra, K., & Bates, B. (2020). *Head first design patterns* (2nd ed.). O'Reilly Media.
<https://www.oreilly.com/library/view/head-first-design/9781492077992/>

Gamma, E. (1995). *Design patterns: Abstraction and reuse of object-oriented design*. European Conference on Object-Oriented Programming.
<https://link.springer.com/chapter/10.1007/BFb0053381>

Gamma, E., Helm, R., Johnson, R., & Vlissides, R. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
<https://www.oreilly.com/library/view/design-patterns-elements/0201633612/>

GeeksforGeeks. (2024). *Adapter Design Pattern y Prototype Design Pattern*.
<https://www.geeksforgeeks.org>

Oracle. (2023). *The Java tutorials: Object-oriented programming concepts*. Oracle Corporation.
<https://docs.oracle.com/javase/tutorial/java/concepts/>

Refactoring Guru. (2024). *Design patterns catalog*.
<https://refactoring.guru/design-patterns>

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.
<https://microservices.io/book>

Spring. (2024). *Spring Boot reference documentation*.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring. (2024). *Spring framework reference documentation*.
<https://docs.spring.io/spring-framework/docs/current/reference/html/>